Scalable Vision Transformers for Embedded Systems

Zachary Swanson and Benjamin S. Riggan

University of Nebraska-Lincoln
Department of Electrical & Computer Engineering,
844 N 16th Street, SEC C290, PO Box 880511
Lincoln, NE 68588-0511
{zswanson2,briggan2}@unl.edu
https://engineering.unl.edu/ece/

Abstract. Deploying face recognition on embedded systems demands models that balance biometric accuracy against strict constraints on latency, memory, and power. This chapter presents a systematic evaluation of vision transformer (ViT) architectures for low-resolution face identification, covering the full pipeline from architecture selection through compression to on-device inference. Starting from 16 model variants across six families—DeiT-3, Swin Transformer, PVT-v2, MobileViT-v2, ConvNeXt, and ResNet—we evaluate parameter-efficient fine-tuning via LoRA, feature-level knowledge distillation with two teacher configurations, structured pruning (head removal and block dropping), and post-training quantization (FP16, INT8) on the TinyFace benchmark. Over 100 resulting configurations are benchmarked on an NVIDIA Jetson Orin Nano using TensorRT. Key findings include: (1) compact transformers outperform larger CNNs on low-resolution inputs—Swin-Tiny (27.5M parameters) with optimally tuned knowledge distillation (α=0.0) achieves 0.616 rank-1 identification rate, exceeding ResNet-200 (62.6M) by +0.148; (2) INT8 quantization reduces model size by 3.8–3.9× with at most 0.002 absolute rank-1 loss; (3) block dropping improves Swin accuracy by up to +0.028 rank-1, confirming depth over-parameterization; and (4) TensorRT compatibility is architecture-dependent, with ConvNeXt, PVT-v2, and MobileViT-v2 failing native conversion due to unsupported normalization patterns. We provide a deployment decision guide mapping model configurations to four common embedded scenarios, from maximum accuracy (DeiT3-B KD, 0.591 rank-1 at 2.6 ms) to minimum memory (DeiT3-S pruned, 0.565 rank-1 at 21.7 MB).

Keywords: Vision Transformers, Face Recognition, Edge AI, Model Compression, Knowledge Distillation, TensorRT

1 Introduction

Vision transformers (ViTs) have emerged as an alternative to convolutional neural networks (CNNs) for computer vision, offering unique benefits especially relevant to embedded systems. Unlike CNNs, which extract features hierarchically via localized receptive fields, ViTs treat an image as a sequence of patches (tokens) and apply self-attention globally. This global receptive field enables ViTs to capture long-range dependencies and holistic context in an image, which can improve robustness for recognition under significant optical distortions or at long standoff distances [34]. For example, transformers can associate spatially distant image regions, potentially outperforming CNNs in scenarios where spatial relationships span the entire image. Modeling global relationships is advantageous in embedded vision applications like aerial imagery or remote biometric recognition, where important cues may be distributed across the frame.

However, ViTs initially came with drawbacks that posed challenges for resource-constrained devices. Early ViT models were extremely large with the Vision Transformer (ViT) model requiring hundreds of millions of parameters and pre-training on massive datasets to match CNN accuracy. For instance, the original ViT-Huge had over 600 million parameters and needed extensive training data (e.g. JFT-300M) to outperform CNNs [7]. At this scale, high model complexity and memory demands are impractical for embedded devices. Moreover, lacking the built-in inductive biases of CNNs (e.g. translation invariance and locality), vanilla transformers tend to require more data or regularization to generalize well [7]. With limited training data, a ViT can struggle to learn stable features, whereas CNNs' local filtering and weight sharing act as useful constraints. These factors historically limited the adoption of ViTs for edge computing devices.

Despite these challenges, recent research demonstrates that vision transformers can be made efficient and scalable for edge deployment. Data-efficient training techniques and compact architectures have narrowed the gap between ViTs and CNNs on limited data [39, 26]. Properly designed transformer models can now achieve competitive accuracy and runtime on embedded hardware. EfficientFormer-L1 attains 79.2% ImageNet top-1 accuracy with only 1.6 ms latency on an iPhone 12 (CoreML), matching the speed of a MobileNet-V2 while outperforming it by over 4% in accuracy [18]. This progress overturns the notion that transformers are too heavy for mobile/embedded applications. ViTs also offer an architectural flexibility (modular self-attention layers) that is amenable to model compression techniques (pruning, quantization, etc.) and parallel acceleration on modern neural processing units (NPUs) [37, 16]. In summary, vision transformers provide discriminative global image representations that are increasingly scalable, and ongoing innovations are making them viable for embedded systems.

This chapter explores the development of ViTs for embedded systems in detail. Section 2 provides an overview of the ViT architecture including explanations of the core components and descriptions of ViT variants and other models explored in this work. Section 3 highlights optimization strategies commonly employed for training transformers including low-rank adaptation (LoRA) and knowledge distillation. Section 4 provides an experimental setup that analyzes performance of the various ViT and CNN models through the lens of a challenging biometrics dataset, TinyFace. The TinyFace dataset features small facial crops, averaging 30×30 pixels, with facial imagery distorted by poor resolution and compression artifacts, motion blur, diverse illumination, and off-axis poses [4]. This dataset was selected because it simulates the type of data that a real-world embedded system may process as part of a surveillance and facial recognition system. Finally, Section 5 presents and analyzes the biometric and computational performance metrics produced by the various models and optimization techniques on the TinyFace dataset. These results provide the reader with valuable data and insights for selecting the correct architecture for their application.

2 Anatomy of Vision Transformers

The ViT architecture in Figure 1 depicts two core components – patch and token generation and the transformer encoder. Both are examined in detail in the following subsections with additional exploration of the multi-head attention mechanism within the transformer encoder.

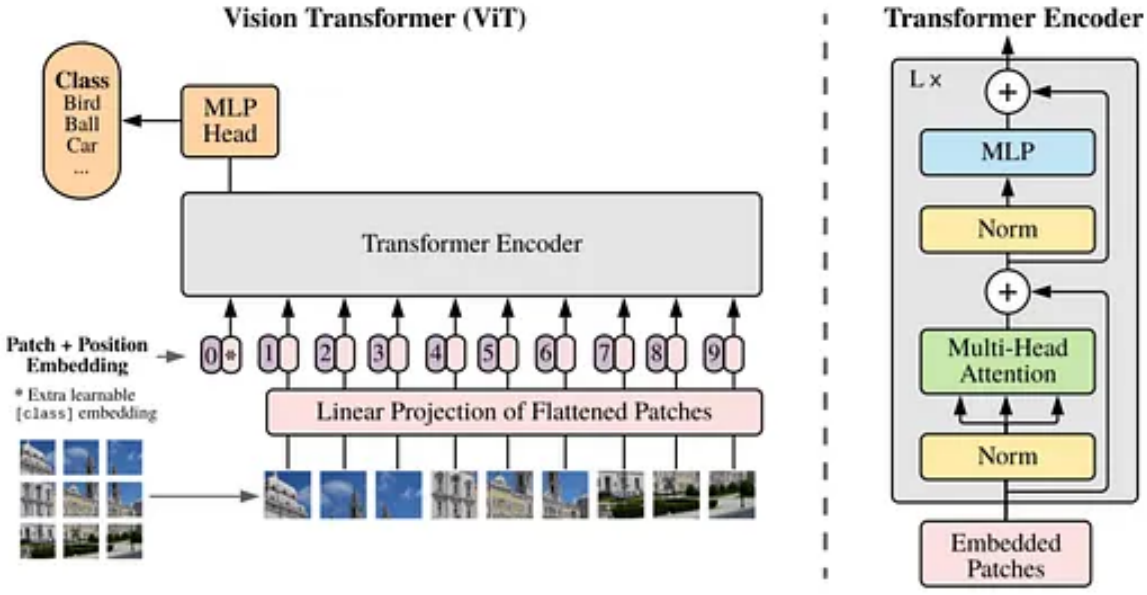


Fig. 1. High-level architecture of a Vision Transformer (ViT) from the original ViT paper from Dosovitskiy et al. [7]. An input image is divided into fixed-size patches, embedded into tokens, and processed by a stack of transformer encoder blocks composed of multi-head self-attention and feed-forward layers.

At a high level, the ViT takes a $P \times P$ image and breaks it into $Q \times Q$ patches such that $(P/Q)^2$ patches are generated, e.g., a 224×224 image split into 16×16 patches generates $(224/16)^2 = 14^2 = 196$

patches. These patches are converted into token vectors and fed to the transformer encoder. The encoder performs self-attention between all patches to weight the vectors based on global "importance" and then performs a non-linear transformation via multilayer perceptron (MLP), also referred to as feed forward network (FFN), to generate a new token representation that is fed to the next encoder layer. The encoder process is repeated through multiple encoder layers effectively refining the token representation before delivering the image representation to a task-specific head. This explanation skips over many crucial details that will be presented in the following sections but provides a clear picture of how images flow through the transformer model.

2.1 Self-Attention and Multi-Head Mechanism

The fundamental component of a transformer is the self-attention layer. In a vision transformer, an image is first split into a sequence of patch embeddings, and a self-attention module then computes interactions between every pair of these patch tokens. Specifically, each token i is projected to a query vector q_i, key vector k_i, and value vector v_i in some latent dimension. The attention operation compares queries and keys via scaled dot-products: $s_{ij} = q_i \cdot k_j/\sqrt{d}$, where d is the token dimension and is applied to appropriately scale the attention scores. A softmax is then applied to $\{s_{ij}\}$ across all tokens j to obtain attention weights α_{ij}, which indicate how much token i attends to token j. These weights are used to compute a weighted sum of the value vectors v_j for token i, updating its representation. Thus, the full attention operation may be represented as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

where Q, K, and V are matrices containing the q, k, and v vectors for all tokens, respectively. This allows information from one token to directly influence another, regardless of spatial distance, enabling the global receptive field of ViTs [41].

Rather than use a single attention head, some transformers employ multi-head self-attention (MHSA). In MHSA, the token embeddings are projected into multiple subspaces via multiple linear layers (heads), and independent self-attention operations are performed in each subspace in parallel. After performing the parallel attentions, the lower-dimensional tokens are concatenated to produce a token of the same dimension as the original input token. Figure 2 provides an example where three heads are applied to three tokens of length 12, i.e., $d = 12$.

In the example of Figure 2, three heads are used and each head processes all three tokens of length 12 ($d = 12$). Thus, the projected dimension is equal to the original token dimension (d) divided by the number of heads (N_H) such that $\frac{d}{N_H} = 4$, as depicted by the smaller tokens to the right of the heads. The smaller tokens have varying shades of color to indicate that each head learns a different representation of the original token. Then, self-attention is performed independently within each subspace, i.e., there are three self-attention operations performed in this example. The output of the independent attention operations are concatenated to produce tokens with the same dimensionality as the original input token.

This design has two benefits: (1) it allows parallel computation of attention, improving throughput on hardware accelerators by performing many smaller matrix multiplies simultaneously; and (2) each head can learn attention in a different representation subspace, so the model can attend to different kinds of features or relationships in parallel [41, 47]. In essence, multi-head attention enables more nuanced and diversified learning, as tokens are compared on distinct portions of their feature descriptors across heads. The outputs of all heads are concatenated and passed through a linear projection to form the final attended representation for each token.

2.2 Transformer Encoder Structure

A vision transformer is constructed by stacking multiple *encoder blocks*, each composed of a multi-head self-attention (MHSA) sub-layer followed by a position-wise feed-forward network (FFN), as shown in Figure 1. Modern ViT architectures typically adopt a *pre-layer normalization (Pre-LN)* design to improve optimization stability, especially for deep models [46]. In this formulation, layer normalization is applied before both the attention and FFN sub-layers, and each sub-layer output is added to its input via a residual skip connection.

Layer normalization is used in place of batch normalization for several reasons. Unlike batch normalization, which normalizes activations across the batch dimension and therefore depends on batch

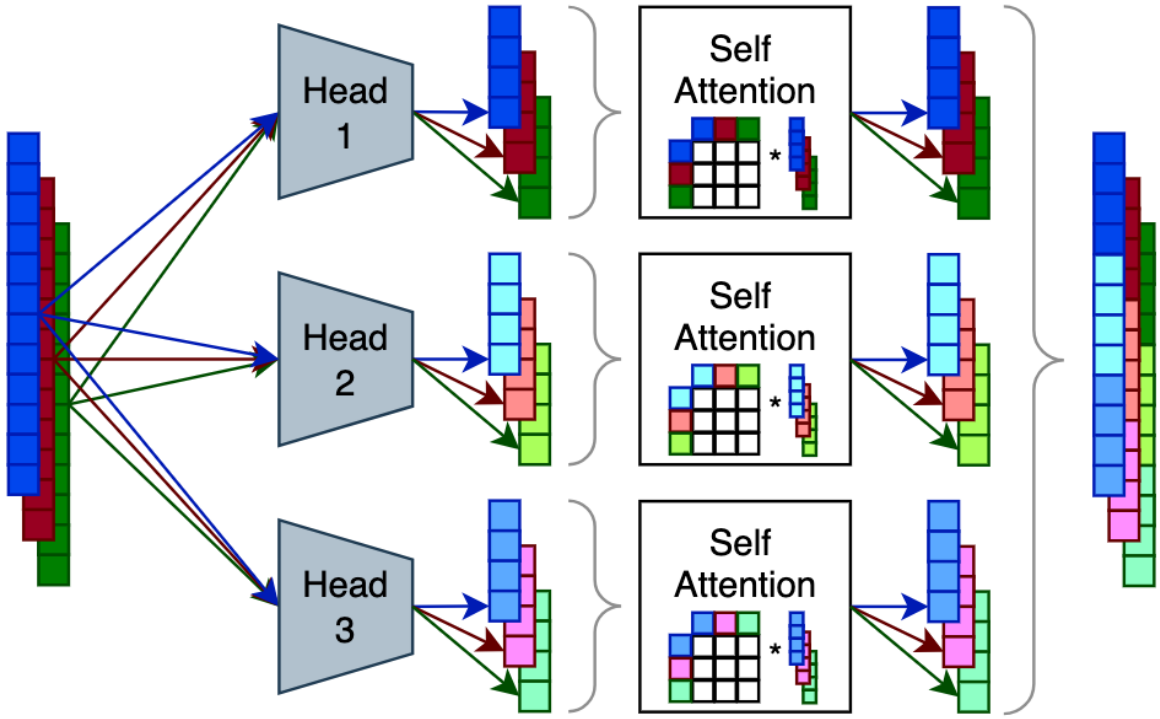


Fig. 2. Example of multi-head self attention on three tokens of length 12 ($d = 12$) where each token is passed to three separate linear projection heads to project the tokens to a smaller dimension (4). Thus, each head ideally learns a different representation of the token indicated by the varying shades. Self-attention is performed in parallel for each subspace and the resulting token vectors are concatenated to produce tokens of the same dimensionality as the original.

statistics, layer normalization operates independently on each token by normalizing across feature dimensions. This makes layer normalization well suited to transformer architectures, where inputs are naturally represented as sequences of tokens and batch sizes may be small or variable. In embedded and low-latency settings, small batch sizes are common due to memory constraints, making batch normalization unstable or ineffective. Moreover, layer normalization is invariant to sequence length and batch size, simplifying deployment and enabling consistent behavior during both training and inference [2].

Formally, given an input token sequence $\mathbf{X} \in \mathbb{R}^{N \times D}$, where N is the number of tokens and D is the token length, an encoder block computes

$$\mathbf{X}' = \mathbf{X} + \mathrm{MHSA}(\mathrm{LN}(\mathbf{X})),$$
$$\mathbf{X}'' = \mathbf{X}' + \mathrm{FFN}(\mathrm{LN}(\mathbf{X}')),$$

where $\mathrm{LN}(\cdot)$ denotes layer normalization, $\mathbf{X}'$ is the intermediate output of the residual MHSA block, and $\mathbf{X}''$ is the output token sequence passed to the next encoder block. This residual formulation preserves gradient flow and enables stable training as depth increases [9].

The FFN is applied independently to each token and consists of two linear transformations, W_1 and W_2, with a non-linear activation σ in between:

$$\mathrm{FFN}(\mathbf{x}) = W_2\, \sigma(W_1 \mathbf{x}),$$

where W_1 expands the feature dimension, W_2 projects it back to the model dimension, and $\sigma = \frac{x}{2}\left(1 + erf\left(\frac{x}{\sqrt{2}}\right)\right)$ is the Gaussian Error Linear Unit (GELU) activation function commonly used in ViT architecture. While the MHSA sub-layer performs global information mixing across tokens, the FFN provides local, channel-wise feature transformation [41].

By stacking L encoder blocks (e.g., $L = 12$ or 24), the model incrementally refines token representations. Earlier layers tend to encode low-level visual patterns and coarse spatial relationships, whereas deeper layers capture higher-level semantic concepts through repeated global attention and nonlinear transformation. This hierarchical abstraction, achieved without explicit convolution, is a defining characteristic of vision transformers and underpins their strong representational capacity [6].

2.3 Tokenization and Position Embeddings

Before entering the transformer encoder, an input image must be converted into a sequence of tokens. In the standard Vision Transformer (ViT) formulation, this tokenization is performed by partitioning the image into fixed-size, non-overlapping patches (e.g., 16×16 pixels) and linearly projecting each flattened patch into a D-dimensional embedding space, typically achieved via convolutional layer with stride equal to kernel size. For example, a 224×224 image divided into 16×16 patches yields $14 \times 14 = 196$ patch tokens, each represented as a vector of dimension $D = 768$ in a ViT-Base model [7].

In addition to patch tokens, a learnable classification token is commonly prepended to the sequence. This token aggregates global information through self-attention and is used as the image-level representation for classification tasks, following a design originally introduced in BERT [5] and later adopted for vision transformers. The resulting input to the transformer encoder is therefore a sequence of $N + 1$ tokens, where N is the number of image patches.However, this is not a requirement and other architectures like the Swin Transformer [24] perform some method of aggregating all the tokens into a single vector after the final layer.

The self-attention mechanism is permutation-invariant and does not encode spatial order by itself; therefore, positional information must be explicitly injected into the token embeddings. Early transformer models employed fixed sinusoidal positional encodings [41], whereas ViT introduced learned absolute positional embeddings [7] that are added to the patch and classification tokens. These embeddings allow the model to associate each token with a specific spatial location in the image grid, enabling meaningful spatial reasoning.

Absolute positional embeddings require special consideration when the input image resolution differs from the pretrained configuration. Since the learned positional embedding table is tied to a fixed number of tokens (e.g., 196 patches for 224×224 inputs), changing the image size alters the number and arrangement of patches, resulting in a mismatch between pretrained positional embeddings and the new token grid. To address this, common strategies include interpolating the pretrained positional embeddings to the new spatial resolution or reinitializing and retraining them. While interpolation often works well in practice, it introduces an implicit assumption of spatial smoothness and may slightly degrade performance, particularly when scaling to very small or highly distorted inputs [32, 3].

To mitigate these issues, several later architectures replace absolute positional embeddings with relative positional encodings or biases. For example, models such as the Swin Transformer [24] incorporate relative position bias directly into the attention computation, allowing the model to encode spatial relationships between tokens independently of the absolute image size. This design improves generalization to arbitrary resolutions and simplifies deployment across varying input dimensions, which is advantageous for embedded and real-world vision systems where input sizes may vary.

2.4 Architectural Survey of Efficient Vision Models

The basic ViT described above is often called a "columnar" or non-hierarchical transformer: it processes tokens at one resolution throughout all layers. Furthermore, the original ViT required a significant amount of training data to achieve competitive performance. Numerous architectural variants have been proposed to improve efficiency and performance for vision tasks; a few major variations and CNN baselines relevant to embedded systems are highlighted below.

Data-Efficient Image Transformer (DeiT, 2021): The DeiT model by Touvron et al. [39] was a crucial step in making transformers practical without massive external data. Architecturally, DeiT is essentially the same as the original ViT (a "base" 12-layer transformer with 86 million parameters) but its innovation lies in the training method. DeiT introduced a knowledge distillation strategy tailored for transformers: a learnable distillation token is appended alongside the class token, and the transformer is trained to predict a teacher CNN's class logits via this token. By using a pre-trained ResNet as teacher and extensive data augmentation and regularization, DeiT achieved 83.1% top-1 accuracy on ImageNet using only ImageNet-1k data (no extra pre-training) [39]. This matches the performance of much larger ViTs that were pre-trained on hundred-million image corpora. The ability to train on a standard dataset with a single GPU machine (in under 3 days) greatly broadened the accessibility of ViTs.

Swin Transformer (2021): Swin Transformer by Liu et al. [24] is a hierarchical ViT designed as a versatile vision backbone for both image classification and dense tasks. Swin introduced shifted local windows for

self-attention. Instead of global attention over all patches, each Swin layer restricts attention to non-overlapping windows (e.g. 7×7 patch windows) which drastically reduces computation. To enable cross-window information flow, every other layer the window partitioning is shifted by a certain offset (hence "shifted" windows), so a patch will attend with neighbors that were in different windows in the previous layer. This scheme achieves linear computational complexity in image size (since window size is fixed) while still allowing distant patches to interact after a few layers. Along with this locality, Swin builds a multi-stage pyramid: patch merging layers between stages reduce the number of tokens (by $2\times$ in each spatial dimension) and increase the feature dimensionality, akin to a CNN downsampling. The result is a transformer that behaves much like a CNN feature hierarchy. The Swin Transformer has two advantages: (1) reduced memory footprint and computational cost by limiting attention to local windows, and (2) compatibility with dense prediction tasks, such as object detection, enabled by the hierarchical image representation, with applications spanning embedded vision systems to surveillance cameras.

Pyramid Vision Transformer (PVT, 2021): PVT by Wang et al. [42] was another early effort to adapt transformers as a general-purpose backbone. PVT introduces a pyramid of transformer stages to handle high-resolution inputs efficiently. In PVT, the image is initially patched into a large number of tokens (e.g. 4×4 patches) for the first stage, but then at each subsequent stage the tokens are merged (concatenating and projecting patches, similar to pooling) to progressively halve the resolution. This yields a pyramid of feature maps at multiple scales, much like a CNN. By the final stage, the token count is greatly reduced, addressing ViT's high memory use on large feature maps. Notably, PVT uses full global attention within each stage (no windowing), but because later stages have few tokens, the overall cost remains manageable. The designers also modified the attention to include a spatial reduction on the attention keys/values for further speedup in high-resolution stages. PVT retains the benefit of both worlds: it has the global receptive field of transformers and the pyramidal structure of CNNs [42].

MobileViT (2022): MobileViT by Mehta & Rastegari [26] is a vision transformer specifically designed for mobile and edge devices. The motivation of MobileViT is to combine the strengths of CNNs and ViTs to create a lightweight model with both local and global feature modeling. The architecture of MobileViT is hybrid: it interleaves convolutional layers with transformer layers in a sequential block. A typical MobileViT block uses a small CNN to extract low-level features, then unfolds those features into a sequence of tokens which are processed by a transformer encoder (attention + FFN), and then folds the tokens back to feature map form which is fused with the convolutional features. This design treats transformers "as convolutions," effectively inserting a global processing unit within a local CNN context. MobileViT also uses very compact dimensions (e.g. embedding dim 144 or 192) and only a few attention layers, making it comparable in size to mobile CNNs [26]. MobileViT demonstrates a feasible recipe: use mostly efficient convolutions, but allocate a fraction of computation to MHSA to boost representation power.

ConvNeXt (2022): ConvNeXt by Liu et al. [25] is a pure convolutional network, included here as a point of comparison because it was directly inspired by the design patterns of transformers. The authors of ConvNeXt asked: if we take a ResNet and modernize it with the tricks that make ViTs successful, can CNNs still compete? They progressively modified a standard ResNet: replacing 3×3 convolutions with larger 7×7 kernels (to increase receptive field, analogous to global attention), using only depthwise separable convolutions (like a transformer's factorized attention), moving the location of normalization layers (using layer normalization instead of batch normalization), and increasing network width and depth based on transformer scaling trends. The outcome: ConvNeXt proved to be on par with transformers [25]. The significance for embedded systems is that CNNs are not obsolete – by adopting transformer-era improvements, CNNs can attain similar accuracy and often with lower inference overhead on certain hardware. Many edge deployments (e.g. DSP or NPU accelerators) are still more optimized for convolutions operations than for MHSA [50, 20, 31]. ConvNeXt offers a compelling alternative when one needs the best of both worlds: it yields transformer-level accuracy but uses standard convolutions layers that may run more efficiently on existing embedded inference engines. In practice, the choice between ConvNeXt and a ViT may come down to the specific hardware and compiler support. ConvNeXt is one indication of how the gap between CNNs and Transformers has narrowed, with both paradigms borrowing ideas from each other.

ResNet (2016): Finally, we consider the classical ResNet architecture by He et al. [9] as a baseline. ResNets introduced the concept of residual learning through skip connections, enabling very deep CNNs

to train effectively. A ResNet's computational structure is a chain of convolutional blocks with occasional downsampling, and its strong hierarchical feature extraction made it the dominant vision backbone of the late 2010s [9]. ResNets are relevant to embedded vision as they are highly optimized in many libraries and hardware accelerators; a ResNet-50 can often run in real-time on mobile NPUs after quantization [14, 8]. However, compared to newer architectures, ResNets have relatively lower accuracy at a given model size and lack the global attention mechanism of ViTs. Throughout this chapter, we use ResNet variants as a conventional baseline to quantify the improvements made by transformer-based models. Some efficient ViTs now exceed ResNet-50's accuracy while being smaller or faster, which is one indication of how much the field has progressed.

3 Optimization Strategies for Embedded Training

Deploying vision transformers on embedded platforms requires more than efficient architectural design—understood as the static layer-wise parameterization and data flow of the model—but also requires training and adaptation strategies that account for constraints on memory, computation, and energy, as well as risks of overfitting. While most embedded devices are ill-suited for large-scale model optimization, careful separation of offline training and lightweight adaptation enables transformer-based models to be deployed effectively in practice. This section outlines common training pipelines for embedded vision transformers, discusses the trade-offs between offline training and on-device adaptation, and introduces optimization techniques like parameter-efficient fine-tuning, knowledge distillation, quantization, and pruning that facilitate deployment under resource constraints. These strategies form the foundation for the TinyFace case study presented in this chapter.

3.1 Training Pipeline and Resource Constraints

In most real-world scenarios, training large vision transformer models is performed offline on high-performance servers or cloud-based GPUs rather than directly on embedded devices [49, 1]. Vision transformers typically require substantial computational resources, including high memory bandwidth and large batch sizes, to train effectively. Embedded platforms, by contrast, are optimized for inference workloads and generally lack the memory capacity, floating-point throughput, and energy budget required for full model training.

As a result, the dominant training pipeline follows a two-stage approach: models are pretrained or fine-tuned offline using hardware with significantly few resource limitations and then deployed on embedded devices exclusively for inference. This paradigm allows developers to exploit advanced optimizers, extensive data augmentation, and long training schedules without imposing additional burdens on the target device. Early embedded machine learning systems largely adhered to this model, focusing on low-latency, on-device inference with pretrained networks [49].

Although on-device training has gained interest due to its potential benefits for privacy, personalization, and domain adaptation, it remains challenging for transformer-based models. Training workloads are significantly more demanding than inference, and even short fine-tuning runs can exhaust memory, increase thermal load, or reduce device lifespan. Consequently, for embedded vision transformers, complete training from scratch is almost always performed offline, with the device serving as a deployment target rather than a training platform. This separation places greater importance on the robustness and adaptability of the trained model to real-world operating conditions [51, 49].

3.2 Offline Training vs. On-Device Adaptation

While offline training is the preferred approach for developing and optimizing vision transformers, many embedded applications require models to adapt over time to new data, users, or environments. In such cases, a hybrid strategy is often employed in which computationally intensive training is performed offline, while limited adaptation is handled either on-device or through periodic updates from a server.

Offline training offers several advantages, including the ability to use large batch sizes, sophisticated optimization algorithms, and diverse data augmentation strategies. Once trained, models can be compressed through techniques such as quantization or pruning and deployed to embedded devices for efficient inference. When updates are needed, retraining or fine-tuning can be performed offline and redistributed to devices, ensuring consistent performance without incurring on-device training costs.

On-device adaptation, when used, is typically restricted to lightweight updates such as incremental learning of new classes, personalization to user-specific data, or calibration to changing sensor conditions [27, 49]. For example, an embedded camera system may adapt a face recognition model to new identities observed in the field. However, these benefits must be weighed against practical constraints: training even a small subset of parameters can significantly increase energy consumption, reduce responsiveness, and stress limited hardware resources. Furthermore, some optimization techniques and adaptive optimizers are poorly supported on embedded NPUs or DSPs [49].

To balance these trade-offs, techniques such as parameter-efficient fine-tuning, federated learning, and split learning have been explored [11, 49]. These methods aim to minimize on-device computation while preserving adaptability, often by limiting the number of trainable parameters or offloading portions of training to external servers. In practice, offline training remains the primary mechanism for developing embedded vision transformers, while on-device adaptation is applied selectively in scenarios where privacy, personalization, or environmental variability justify the additional resource cost.

3.3 Parameter-Efficient Tuning (LoRA)

When adapting large vision transformers to a new task or dataset, updating all model parameters is often unnecessary and computationally expensive. Parameter-efficient fine-tuning (PET) methods address this issue by restricting training to a small subset of parameters while keeping the pretrained backbone fixed. Low-Rank Adaptation (LoRA) is a prominent and effective PET technique that has been successfully applied to vision transformers.

LoRA operates by freezing the original model weights and injecting additional trainable low-rank matrices into selected layers, most commonly the linear projections within multi-head self-attention (MHSA) and, in some cases, the feed-forward networks (FFNs). Rather than updating a full weight matrix $W \in \mathbb{R}^{d\times d}$, LoRA introduces an additive update

$$W' = W + \Delta W, \quad \Delta W = AB^\top,$$

where $W' \in \mathbb{R}^{d\times d}$ is the adapted weight matrix and $A \in \mathbb{R}^{d\times r}$ and $B \in \mathbb{R}^{d\times r}$ are trainable matrices of rank $r \ll d$. The original weight W remains frozen, and only the low-rank factors A and B are optimized. Typical rank values such as $r = 4$ or $r = 8$ introduce only a small fraction of additional parameters while allowing sufficient expressive capacity for task adaptation.

This formulation significantly reduces the number of trainable parameters and associated optimizer state, often to well under 1% of the full model size. Hu et al. [12] demonstrated that LoRA can achieve performance comparable to full fine-tuning across a range of tasks while dramatically lowering training cost. From an optimization standpoint, LoRA also improves stability by constraining updates to a low-dimensional subspace, reducing the risk of overfitting or catastrophic forgetting, as demonstrated in works like PETALface [28].

For embedded and resource-constrained settings, the advantages of LoRA are particularly compelling. Fine-tuning can be performed with smaller batch sizes, reduced memory consumption, and faster convergence, making it feasible to adapt large pretrained models using modest hardware. However, it should be noted that LoRA does not reduce the memory footprint training because all hidden states are required to train the new low-rank matrices [43]. Because the backbone weights remain unchanged, LoRA preserves the model's original representational capabilities while learning task-specific corrections. In low-resolution face recognition scenarios, for example, LoRA enables a transformer pretrained on high-quality facial imagery to adapt to datasets such as TinyFace without degrading its performance on higher-resolution domains [28]. Recent works such as PETALface and FaceMoE [28, 29] further demonstrate that conditioning or gating LoRA updates can improve robustness across varying image qualities, though the underlying low-rank adaptation principle remains broadly applicable.

From a deployed embedded systems perspective, LoRA also dramatically reduces the size of the payload required to update model weights over the network. Because the frozen backbone is identical across all devices, only the low-rank factors A and B need to be transmitted when distributing a new adaptation. For a Swin-Tiny backbone (27.5 M FP32 parameters $\approx$ 110 MB), a rank-r=8 LoRA update applied to the attention projections contains well under 1% of the full parameter count—typically on the order of $\sim$100 KB—a 1000$\times$ reduction in over-the-air transfer size relative to shipping a full fine-tuned checkpoint. This makes LoRA particularly attractive for fleets of bandwidth-constrained or intermittently-connected edge devices, where periodic model refreshes (e.g., personalization to a new user, adaptation to a new sensor batch, or domain calibration) can be delivered as small "patch" files that the device merges into the resident backbone at load time without ever downloading a fresh full-precision model.

3.4 Knowledge Distillation

Knowledge distillation (KD) is a widely used technique for transferring knowledge from a large, high-capacity *teacher* model to a smaller or more efficient *student* model. Rather than training the student solely on hard class labels, distillation encourages the student to match the teacher's output distribution, intermediate representations, or both. This process provides additional supervisory signals that can improve convergence, generalization, and sample efficiency, particularly when training compact models.

In the context of vision transformers, knowledge distillation has played a critical role in reducing data and compute requirements. The Data-efficient Image Transformer (DeiT) demonstrated that ViTs could be trained effectively on ImageNet-1k without large-scale pretraining by distilling knowledge from a convolutional neural network teacher. DeiT introduced a learnable *distillation token* alongside the standard classification token, allowing the transformer to explicitly model the teacher's predictions through an auxiliary supervision path. This design enabled ViTs to achieve competitive accuracy with CNNs while significantly reducing the reliance on massive external datasets [39].

Beyond output-level distillation, several works have explored feature-level and attention-level distillation for transformers. In these approaches, the student is encouraged to align intermediate feature maps, attention matrices, or token embeddings with those of the teacher. Such strategies are particularly effective when distilling from a larger transformer into a smaller one, as they transfer structural inductive biases and attention patterns that are not captured by class probabilities alone. For embedded deployment, this enables smaller transformer models to inherit much of the representational power of larger backbones while maintaining lower latency and memory footprints [48, 45].

From an embedded systems perspective, knowledge distillation is especially attractive because it shifts computational burden entirely to the training phase. The teacher model is used only offline, and the distilled student incurs no additional inference cost. This makes KD complementary to architectural efficiency and parameter-efficient fine-tuning methods such as LoRA. In practice, distillation is often combined with model compression techniques, including quantization or pruning, to further reduce inference cost without significant accuracy degradation.

3.5 Quantization and Pruning

In addition to architectural design and training-time optimizations, post-training model compression techniques such as quantization and pruning are essential for deploying vision transformers on embedded platforms. These methods aim to reduce memory footprint, computational cost, and energy consumption while preserving as much predictive performance as possible.

Quantization: Quantization reduces the numerical precision of model parameters and activations, typically from 32-bit floating point (FP32) to lower-bit representations such as 16-bit floating point (FP16) or 8-bit integer (INT8). On embedded hardware, lower-precision arithmetic is often significantly faster and more energy-efficient, and it allows models to fit within limited on-chip memory [35]. For vision transformers, quantization can be applied to linear projections in attention layers, feed-forward networks, and embedding layers [19]. Post-training quantization (PTQ) is attractive for deployment because it does not require retraining the model; instead, calibration data is used to estimate appropriate scaling factors [38]. However, transformers can be more sensitive to quantization error than CNNs, particularly in the attention softmax and layer normalization operations [33]. Quantization-aware training (QAT) addresses this issue by simulating quantization effects during training, allowing the model to adapt its parameters to reduced precision [13]. In practice, many embedded deployments adopt mixed-precision schemes, retaining higher precision for numerically sensitive operations (e.g., softmax and normalization) while quantizing the bulk of linear layers to INT8. Such hybrid approaches often achieve substantial speedups with minimal accuracy loss [21].

Pruning: Pruning reduces model complexity by removing parameters or structures that contribute little to overall performance. In unstructured pruning, individual weights with small magnitude are set to zero, yielding sparse matrices. While this can significantly reduce parameter count, unstructured sparsity is difficult to exploit on general-purpose hardware without specialized sparse kernels. As a result, structured pruning is often preferred for embedded systems [15].

Structured pruning removes entire components such as attention heads, channels, or even transformer layers [35]. For vision transformers, common strategies include pruning redundant attention heads, reducing the width of feed-forward networks, or dropping entire encoder blocks in deeper layers. Because

the transformer encoder is modular and highly repetitive, it lends itself well to such coarse-grained pruning [17]. Structured pruning produces dense sub-networks that map efficiently onto existing hardware accelerators [35].

Embedded Deployment Considerations: From an embedded systems perspective, the effectiveness of quantization and pruning depends strongly on hardware support and software tooling [30]. Many NPUs and DSPs provide native INT8 or mixed-precision acceleration but offer limited support for sparsity[35]. Consequently, structured pruning combined with quantization-aware training is often the most practical compression strategy for real-world deployment [22, 15]. When carefully applied, these techniques enable vision transformers to meet strict latency, memory, and power constraints while retaining much of their original representational strength [19].

4 Experimental Setup

This section describes the dataset, preprocessing choices, training protocol, and evaluation methodology used to assess the performance and deployability of vision transformer models in an embedded setting. The experimental design emphasizes both biometric accuracy and computational efficiency, reflecting the dual objectives of recognition performance and practical deployment.

4.1 Dataset Selection: TinyFace

The TinyFace dataset was selected as the primary benchmark for this study due to its emphasis on extremely low-resolution facial imagery. TinyFace contains over 160,000 face images spanning 5,139 identities, with an average native resolution of approximately 30×30 pixels. Images exhibit significant degradation due to limited spatial resolution, compression artifacts, pose variation, and illumination changes. These characteristics make TinyFace a particularly challenging dataset for face recognition and a strong proxy for embedded vision scenarios such as long-range surveillance, low-bandwidth sensing, and resource-constrained edge devices [4].

Unlike many standard face recognition benchmarks that assume high-quality facial crops, TinyFace stresses a model's ability to extract discriminative features from minimal visual information. This aligns well with the goals of this work, which seeks to evaluate whether scalable vision transformers can remain effective when both input resolution and deployment hardware are severely constrained.

4.2 Image Resolution and Preprocessing

All TinyFace images were resized to a fixed spatial resolution of 96×96 pixels prior to being passed to the models. This resolution was chosen as a compromise between preserving spatial detail and avoiding excessive upsampling of the original low-resolution images. Given an average native resolution of approximately 30×30 pixels, more aggressive resizing (e.g., 112×112 or 224×224) was found to introduce substantial interpolation artifacts without meaningful gains in discriminative information.

From a transformer perspective, a 96×96 input resolution provides a sufficient number of patch tokens to enable effective self-attention while keeping computational cost manageable. For example, with a patch size of 16×16, this resolution yields $6 \times 6 = 36$ tokens, striking a balance between spatial expressiveness and efficiency. This choice also aligns with embedded deployment goals, where minimizing token count directly reduces attention complexity and memory usage.

Training augmentations included random horizontal flips, random rotation ($\pm 15°$), color jitter (brightness, contrast, saturation, and hue), and random affine translations ($\pm 10\%$). All images were normalized using ImageNet statistics (mean $= [0.485, 0.456, 0.406]$, std $= [0.229, 0.224, 0.225]$) [36]. Evaluation used only resizing and normalization without augmentations.

4.3 Model Architectures and Configurations

Six architecture families were evaluated, spanning pure transformers (DeiT-3, Swin, PVT-v2), hybrid models (MobileViT-v2), modernized CNNs (ConvNeXt), and classical CNNs (ResNet). Table 1 summarizes the model variants and their parameter counts.

All models were initialized from ImageNet-1k [36] pretrained checkpoints provided by the `timm` library [44] and adapted to the TinyFace classification task by replacing the classification head. Two training modes were investigated for each architecture: (1) *baseline* training, in which all backbone parameters are updated, and (2) *LoRA* fine-tuning, in which the backbone is frozen and only low-rank adaptation matrices are trained.

	Base		Small		Tiny	
	Params	Layers	Params	Layers	Params	Layers
ConvNeXT	87.6	36	49.5	36	27.8	18
DeiT-3	85.8	12	38.3	12	21.7	12
MobileViT	-	-	-	-	17.4	9
PVT	81.4	52	44.7	28	24.8	16
ResNet	62.6	66	42.5	33	23.5	16
Swin	86.7	24	48.8	24	27.5	12

Table 1. Architecture variants evaluated in this study with approximate parameter counts (millions) and total transformer/convolutional block counts. DeiT-3 Base/Small/Tiny correspond to DeiT3-B/M/S; PVT Base/Small/Tiny to PVT-v2-B5/B3/B2; ResNet Base/Small/Tiny to ResNet-200/101/50. MobileViT-v2-200 layer count reflects the 9 linear transformer blocks across 3 hybrid stages. All models use ImageNet-1k pretrained weights from the `timm` library [44].

4.4 Training Protocol

Prior to final training runs, Bayesian hyperparameter sweeps were conducted to identify stable and effective configurations for each model family. These sweeps explored learning rate, weight decay, head margin, and LoRA-specific parameters (rank, alpha, dropout), with the goal of identifying hyperparameter regions that produced consistent convergence.

Baseline Training. For baseline experiments, all backbone parameters were trainable. Models were trained for 100 epochs using the AdamW optimizer with a cosine annealing learning rate schedule. Batch sizes of 512 were used with mixed-precision (FP16) training. The AdaFace [28] angular margin head was used for all models, with margin and scale parameters tuned per family via the hyperparameter sweeps. The overall training loss is the cross-entropy between the margin-scaled logits and the identity labels.

LoRA Fine-Tuning. For LoRA experiments, the pretrained backbone was frozen and low-rank adaptation matrices were injected into the query, key, value, and output projection layers of each attention block. Following Hu et al. [12], the LoRA update is

$$W' = W + \frac{\alpha}{r} \cdot BA,$$

where $A \in \mathbb{R}^{d\times r}$ and $B \in \mathbb{R}^{r\times d}$ are trainable low-rank factors, r is the rank, and α is a scaling coefficient. Typical swept values were $r \in \{4, 8, 16, 24, 32\}$ and $\alpha \in \{16, 24, 32, 96, 120\}$. LoRA models were trained for 50 epochs with a batch size of 8, a cosine schedule with 8 warmup epochs, and gradual unfreezing of backbone stages every 5 epochs. The learning rate floor was set to 0.1% of the peak value.

4.5 Knowledge Distillation Setup

To evaluate the impact of knowledge distillation on student model performance, a feature-level distillation framework was implemented. The combined training loss is defined as

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{CE}}(\text{logits}, y) + (1 - \alpha) \cdot \mathcal{L}_{\text{KD}}, \tag{2}$$

where $\mathcal{L}_{\text{CE}}$ is the standard cross-entropy loss with the AdaFace head, $\mathcal{L}_{\text{KD}}$ is the cosine embedding loss between L2-normalized teacher and student feature vectors, and $\alpha \in [0, 1]$ controls the relative weighting. A trainable linear projection layer maps the student embedding dimension to the teacher's 512-dimensional space when dimensions differ.

Two teacher models, both pretrained on the large-scale WebFace4M dataset, were evaluated:

- **CVLFace**: A ViT-Base model with AdaFace head (114.9M parameters), accepting 112×112 inputs and producing 512-dimensional L2-normalized embeddings. This represents a cross-architecture teacher (ViT $\rightarrow$ Swin/DeiT student).
- **PETALface**: A custom 3-stage Swin Transformer with ArcFace head (213.8M parameters), accepting 120×120 inputs and producing 512-dimensional L2-normalized embeddings [28]. This represents a same-architecture teacher for Swin students.

Both teachers are substantially larger than the student models evaluated in this study (17.4M–87.6M parameters), and their high accuracy stems in part from pretraining on WebFace4M (4.2 million identities), which is orders of magnitude larger than TinyFace. These factors—large model size, high input resolution (112×112 or 120×120 vs. the student's 96×96), and extensive pretraining data—make the teachers impractical for direct deployment on embedded platforms. Knowledge distillation therefore serves as the mechanism to transfer the teachers' representational capacity to smaller, deployable student architectures.

Each teacher was evaluated in two modes: *frozen* (original pretrained weights) and *fine-tuned* (adapted to TinyFace before serving as teacher). During KD training, student images (96×96, ImageNet-normalized) are un-normalized, resized to the teacher's input resolution, and re-normalized to $[-1, 1]$ before being passed to the teacher. The default distillation weight was $\alpha = 0.7$; an ablation study across $\alpha \in \{0.0, 0.1, 0.3, 0.5, 0.7, 0.8, 0.9, 1.0\}$ was conducted to identify optimal mixing ratios.

KD student models were trained with the same protocol as LoRA models: 50 epochs, batch size 8, cosine schedule with warmup, and AdamW optimizer. Both baseline (all parameters trainable) and LoRA training modes were evaluated with each teacher configuration.

4.6 Evaluation Methodology

All models were evaluated using the standard TinyFace gallery–probe protocol. Probe images are matched against a gallery of enrolled identities using cosine similarity of the extracted embeddings. The primary evaluation metrics include:

- **Rank-k identification rate**: The fraction of probe images for which the correct identity appears within the top k gallery matches, reported for $k = 1$ through 10.
- **Mean average precision (mAP)**: The mean of per-query average precision scores across all probe images.
- **Area under the curve (AUC)**: The area under the receiver operating characteristic (ROC) curve.

Rank-1 identification rate is used as the primary metric for model selection and comparison, as it directly reflects the practical accuracy of a single-shot identification system. All experiments used a fixed random seed of 73 for reproducibility.

4.7 ONNX Export and Quantization

To evaluate deployment-readiness, all trained models were exported to the ONNX (Open Neural Network Exchange) format and quantized to reduced-precision representations. ONNX provides a hardware-agnostic inference format supported by embedded runtime engines including ONNX Runtime, TensorRT, and vendor-specific NPU compilers.

Each model was exported from its PyTorch Lightning checkpoint with `torch.onnx.export` at opset version 18 with dynamic batch axes, producing FP32 ONNX graphs. For LoRA-adapted models, low-rank matrices were algebraically merged into the base weights prior to export (i.e., $W' = W + \frac{\alpha}{r}BA$), yielding a standard dense model with no adapter overhead at inference. For KD-trained models, only the student backbone was exported; the teacher and projection layers were discarded.

Two quantization schemes were applied:

- **FP16**: Static weight conversion from FP32 to IEEE 754 half-precision via onnxconverter-common, reducing model size by approximately $2\times$.
- **INT8**: Dynamic quantization of linear layer weights (`MatMul` and `Gemm` operators) to 8-bit integers via ONNX Runtime's `quantize_dynamic`, targeting approximately $4\times$ size reduction.

All FP32 ONNX exports were validated against the original PyTorch models by computing cosine similarity between output embeddings on a shared calibration set, requiring > 0.999 similarity. Each quantized model was then independently evaluated on the TinyFace test set using ONNX Runtime inference (CPU) to measure the actual impact of reduced precision on recognition accuracy.

4.8 Structured Pruning

To investigate further compression beyond quantization alone, structured pruning was applied systematically across all four architecture families. Five source checkpoints were selected to span both architectures and training regimes: Swin-T Baseline, Swin-T KD Best (α=0.0), DeiT3-S Baseline, Swin-B Baseline, and DeiT3-B KD Best (α=0.1). Two structured pruning strategies were evaluated:

Attention Head Pruning. For each multi-head self-attention (MHSA) layer, a per-head importance score is computed as the L2 norm of the concatenated query, key, value, and output projection weight slices associated with that head:

$$I_h = \|W_h^Q\|_2 + \|W_h^K\|_2 + \|W_h^V\|_2 + \|W_h^{\text{proj}}\|_2,$$

where $W_h^Q \in \mathbb{R}^{d_k \times d}$ denotes the projection weights for head h of dimension $d_k = d/N_H$. Heads with the lowest importance scores are removed, and the corresponding weight slices are physically deleted from the QKV and output projection matrices. This produces a structurally smaller model with fewer attention heads per layer. Pruning ratios of 10%, 25%, and 50% of heads were evaluated.

Block Dropping. Entire transformer encoder blocks are removed from the model, following a last-first ordering scheme: the deepest blocks are removed first, as later layers in transformers tend to exhibit greater redundancy [17]. For DeiT3-S ($L = 12$) and DeiT3-B ($L = 12$), 2 and 3 blocks were removed respectively; for Swin-T ($L = 12$ across 4 stages) and Swin-B ($L = 18$ across 4 stages), 2 and 5 blocks were removed from the later stages.

All pruned models were fine-tuned for 10 epochs with all parameters unfrozen, using AdamW (lr = 3×10^{-5}, weight decay = 0.05) with cosine annealing, to recover accuracy lost during pruning. Pruned models were then exported to ONNX and quantized to FP16 and INT8 using the same pipeline described in Section 4.7, yielding 72 pruned+quantized model variants (24 pruned configurations $\times$ 3 precisions).

5 Performance Analysis

This section presents the experimental results across six lines of evaluation: (1) baseline architecture comparison, (2) parameter-efficient fine-tuning via LoRA, (3) knowledge distillation with multiple teacher configurations, (4) an ablation study on the distillation loss mixing coefficient, (5) ONNX export and quantization of selected models, and (6) structured pruning combined with quantization. Results are analyzed through the lens of both recognition accuracy and model efficiency to inform deployment decisions for embedded systems.

5.1 Baseline Architecture Comparison

Sixteen model variants spanning six architecture families were trained on TinyFace using the baseline protocol described in Section 4.4. Figure 3 presents the rank-1 identification rates, and Figure 4 shows the relationship between model size and accuracy.

Several observations follow from the baseline comparison. First, vision transformers outperform CNNs on this low-resolution task. DeiT3-Base achieves the highest rank-1 rate of 0.583, followed by DeiT3-Medium (0.570) and Swin-Tiny (0.562). All three ResNet variants rank at the bottom; ResNet-200 (0.468) performs worst despite having 62.6M parameters. One plausible explanation is that global self-attention provides an advantage when discriminative information must be extracted from severely limited spatial resolution, as is the case with TinyFace's 30×30 average native input.

Second, larger models within a family do not always perform better. Swin-Tiny (27.5M parameters, rank-1 = 0.562) outperforms Swin-Base (86.7M, 0.550), and DeiT3-Medium (38.3M, 0.570) is roughly on par with DeiT3-Base (85.8M, 0.583) at less than half the parameter count. For ResNets, scaling from 50 to 200 layers actually *decreases* performance (0.476 $\rightarrow$ 0.468), indicating that additional capacity does not compensate for the architectural limitations of purely local feature extraction at these resolutions. This inverse scaling matters for embedded deployment: smaller transformer variants achieve better accuracy and lower computational cost at the same time.

Third, ConvNeXt models, despite incorporating many transformer-inspired design principles, perform only slightly above their ResNet counterparts and substantially below DeiT-3 and Swin. ConvNeXt-Base (87.6M) achieves 0.543, lower than Swin-Tiny at one-third the parameters. At the extremely low input resolutions characteristic of TinyFace, explicit self-attention—not merely large receptive fields or modern training practices—appears to be the dominant factor.

Figure 4 also plots the teacher models used in subsequent knowledge distillation experiments (Section 4.5). The CVLFace ViT-Base teacher (114.9M parameters) achieves rank-1 rates of 0.716 (frozen) and 0.781 (fine-tuned), while the PETALface Swin teacher (213.8M parameters) achieves 0.424 (frozen) and 0.663 (fine-tuned). Although the teachers offer substantially higher accuracy—particularly after fine-tuning on TinyFace—their parameter counts of 114.9M and 213.8M exceed the largest student models

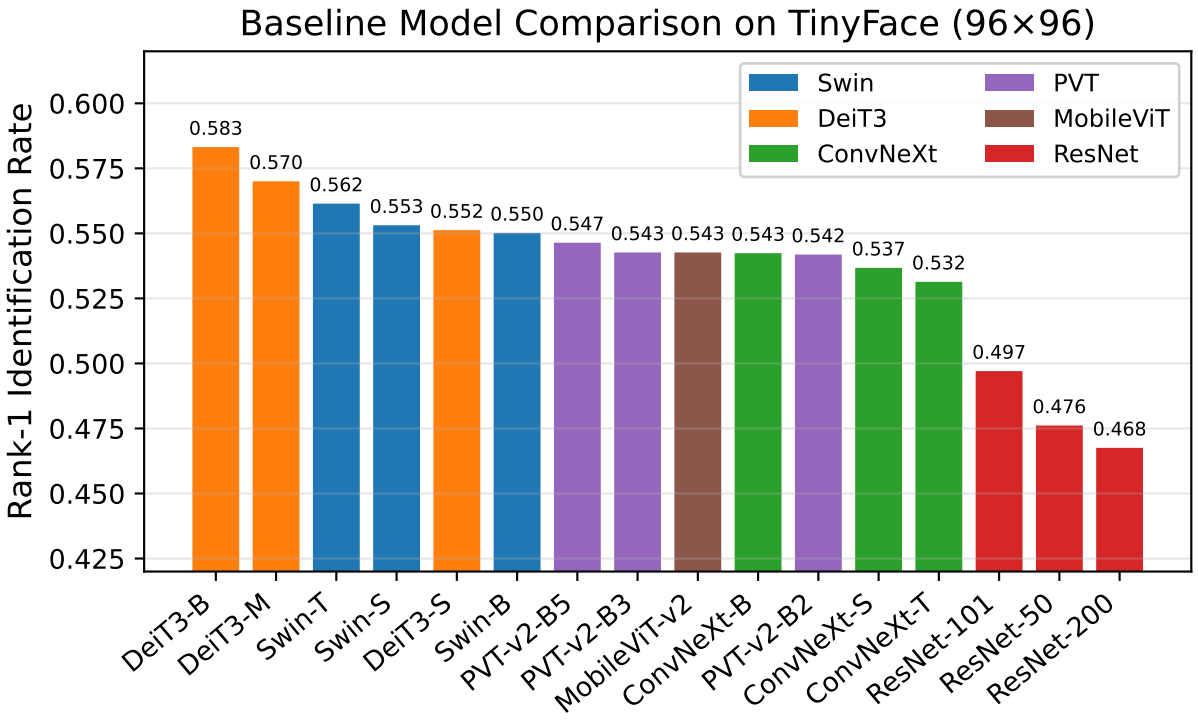


Fig. 3. Rank-1 identification rate on the TinyFace test set for all 16 baseline model variants, sorted by performance. On this low-resolution dataset, vision transformers (DeiT-3, Swin) outperform CNNs (ResNet, ConvNeXt) and hybrid architectures.

by 30–150%, placing them well outside the feasible range for embedded deployment. This performance–size gap motivates the use of knowledge distillation to transfer teacher-level representational capacity to compact student architectures.

5.2 Parameter-Efficient Fine-Tuning with LoRA

Following the baseline comparison, the top-performing architecture families (Swin and DeiT-3) were fine-tuned using LoRA as described in Section 4.4. Figure 5 compares baseline and LoRA results, and Figure 6 presents CMC curves for selected top models alongside a ResNet-50 reference.

The LoRA results show an architecture-dependent response to parameter-efficient fine-tuning. For Swin models, LoRA consistently improves accuracy over the baseline: Swin-Tiny improves from 0.534 to 0.562 (+0.028 rank-1), Swin-Small from 0.539 to 0.554 (+0.015), and Swin-Base from 0.526 to 0.536 (+0.010). DeiT-3 models exhibit the opposite trend: LoRA degrades performance for DeiT3-Small (0.552 → 0.527, −0.025), DeiT3-Medium (0.570 → 0.533, −0.037), and DeiT3-Base (0.583 → 0.550, −0.033).

This divergence may be attributed to architectural differences in how Swin and DeiT-3 process information. Swin Transformer's hierarchical structure with local window attention may benefit from the regularization effect of LoRA's low-rank constraint, which prevents overfitting while still adapting attention patterns. DeiT-3's columnar architecture with global attention across all layers may already be well-adapted to the task via full fine-tuning, and constraining updates to a low-rank subspace may limit its ability to make the necessary adjustments to its global attention patterns for this domain.

The CMC curves in Figure 6 further illustrate these dynamics. DeiT3-Base with LoRA achieves the highest cumulative identification rates, surpassing even the DeiT3-Base baseline at higher ranks ($k \geq 2$). Both DeiT3-Base configurations substantially outperform the ResNet-50 baseline, which lags by approximately 0.10 in rank-1. The consistent gap between transformer and CNN models across all ranks confirms that the advantage of self-attention generalizes beyond rank-1 accuracy.

5.3 Knowledge Distillation Results

Knowledge distillation experiments were conducted for all Swin and DeiT-3 student models using both CVLFace and PETALface teachers in frozen and fine-tuned modes. Table 2 summarizes the key results.

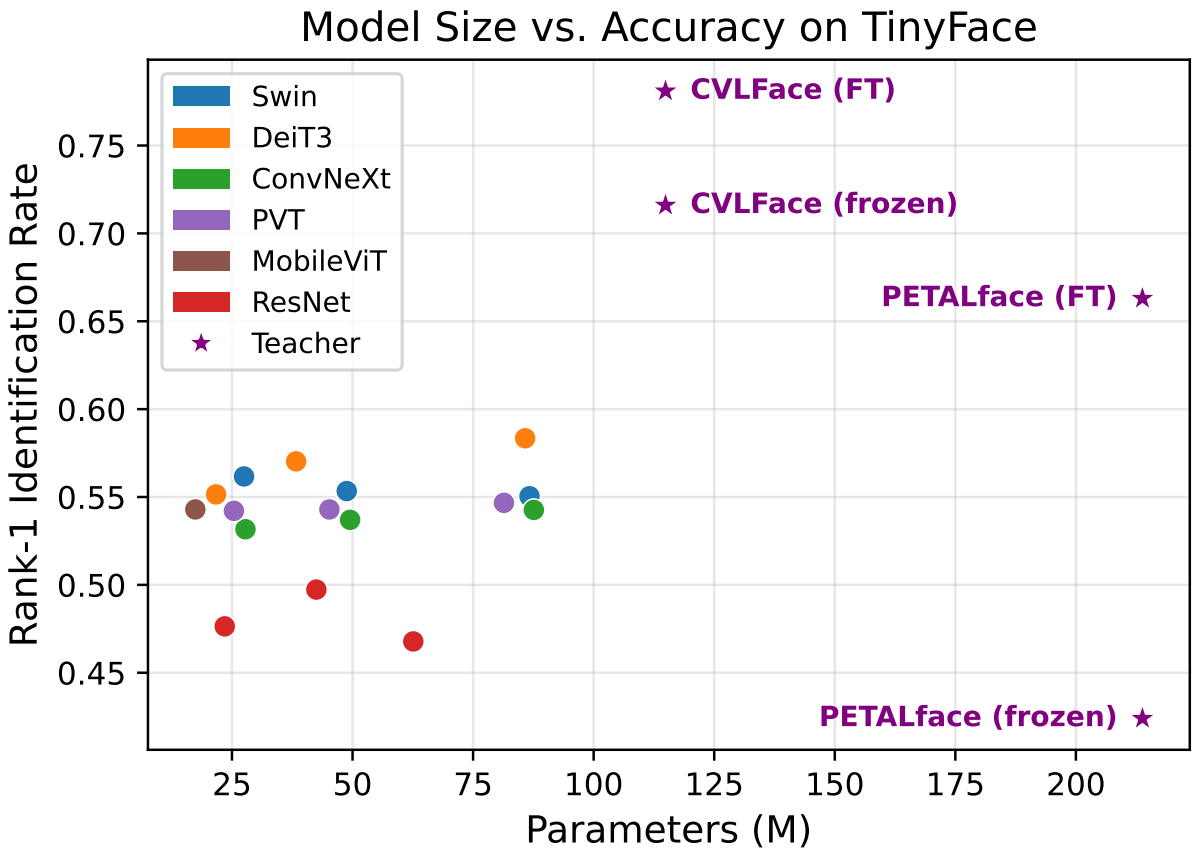


Fig. 4. Model parameter count versus rank-1 identification rate. Student models (circles) range from 17.4M to 87.6M parameters, while teacher models (stars) used for knowledge distillation are substantially larger (CVLFace 114.9M, PETALface 213.8M). The teachers' size and input resolution requirements make them impractical for embedded deployment, motivating the distillation approach studied in Section 4.5.

Table 2. Knowledge distillation results (rank-1) compared to standalone training. *Best B/L* refers to the better of baseline or LoRA for each model. The best KD result across all teacher configurations is shown. Δ indicates the change from the best non-KD result to the best KD result. Default $\alpha = 0.7$.

Model	Best B/L	Best KD	Best Config	Δ
Swin-T	0.562	0.567	PETALface frozen, LoRA	+0.005
Swin-S	0.554	0.559	CVLFace FT, LoRA	+0.004
Swin-B	0.536	0.536	CVLFace FT, Direct	+0.000
DeiT3-S	0.552	0.543	CVLFace frozen, Direct	−0.009
DeiT3-M	0.570	0.559	CVLFace FT, Direct	−0.011
DeiT3-B	0.583	0.581	CVLFace FT, Direct	−0.003

Several findings emerge from the distillation study. First, KD with the default $\alpha = 0.7$ provides only marginal improvements for Swin students (+0.000 to +0.005 rank-1) and slight degradation for DeiT-3 students (−0.003 to −0.011). The higher-accuracy teacher (CVLFace ViT-Base, rank-1 = 0.716 frozen, 0.781 fine-tuned) does not translate its substantial performance advantage into student accuracy at this default mixing ratio. On this dataset the bottleneck appears to lie in the student's capacity to extract discriminative features from a 96×96 input rather than in the quality of the supervisory signal.

Second, teacher fine-tuning does not reliably improve student performance. Fine-tuning dramatically improves the teacher's own accuracy (CVLFace: 0.716 → 0.781; PETALface: 0.424 → 0.663), but the resulting KD students perform inconsistently, sometimes better and sometimes worse than those trained with frozen teachers. This challenges the common assumption that a stronger teacher necessarily produces a stronger student.

Third, the teacher architecture matters but not in the expected direction. Despite PETALface achieving substantially lower accuracy on TinyFace (rank-1 = 0.424 frozen vs. CVLFace's 0.716), it occasionally

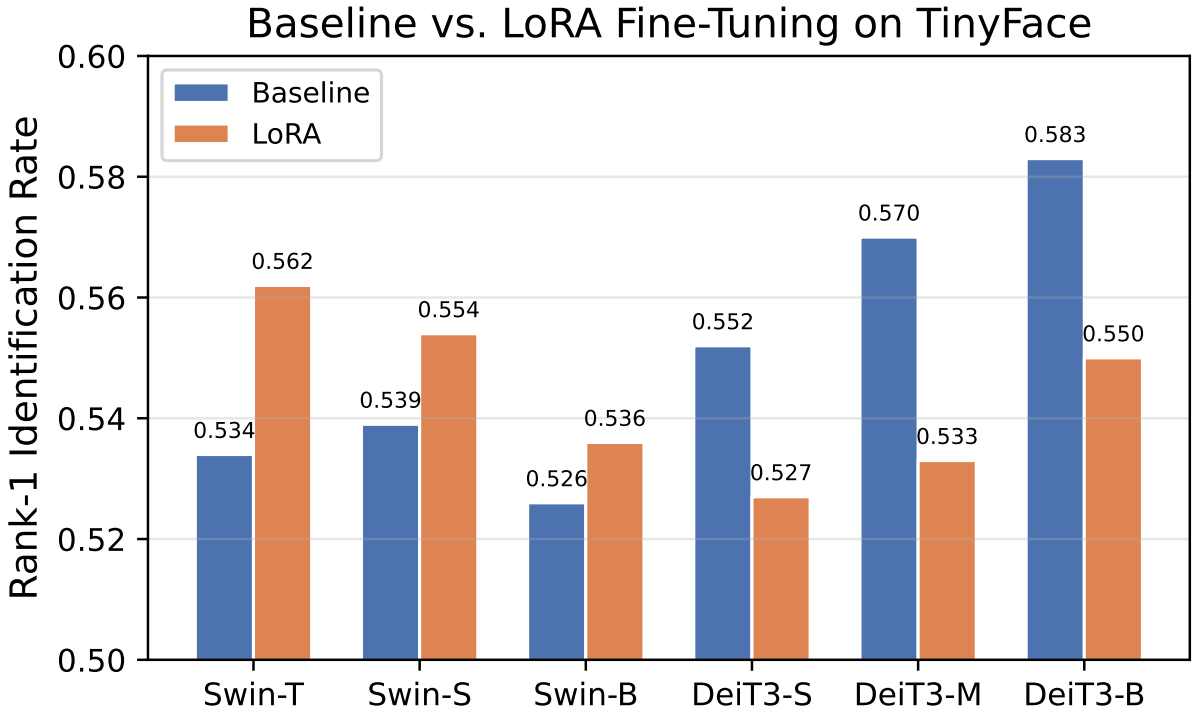


Fig. 5. Comparison of baseline (full fine-tuning) and LoRA fine-tuning across Swin and DeiT-3 model variants. LoRA improves Swin models but degrades DeiT-3 performance, revealing an architecture-dependent response to parameter-efficient adaptation.

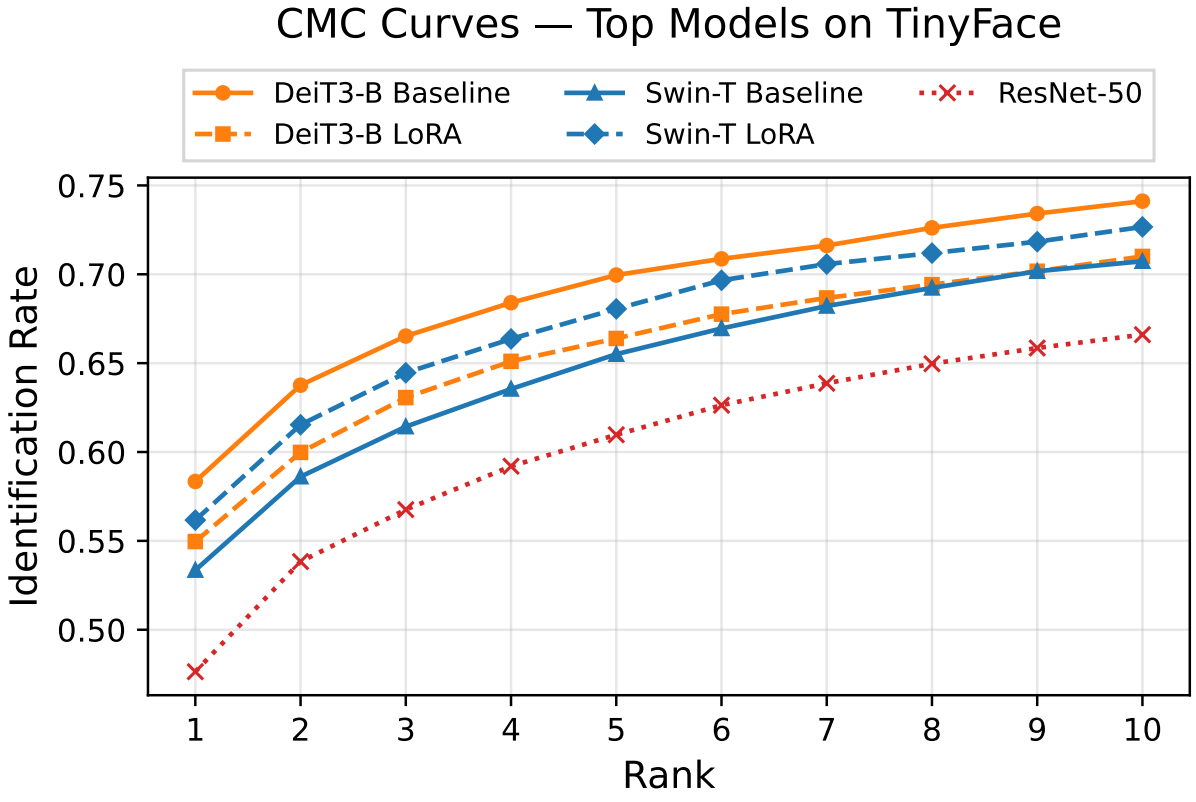


Fig. 6. Cumulative match characteristic (CMC) curves for the top-performing models compared to a ResNet-50 baseline. DeiT3-Base with LoRA achieves the highest identification rates across all ranks.

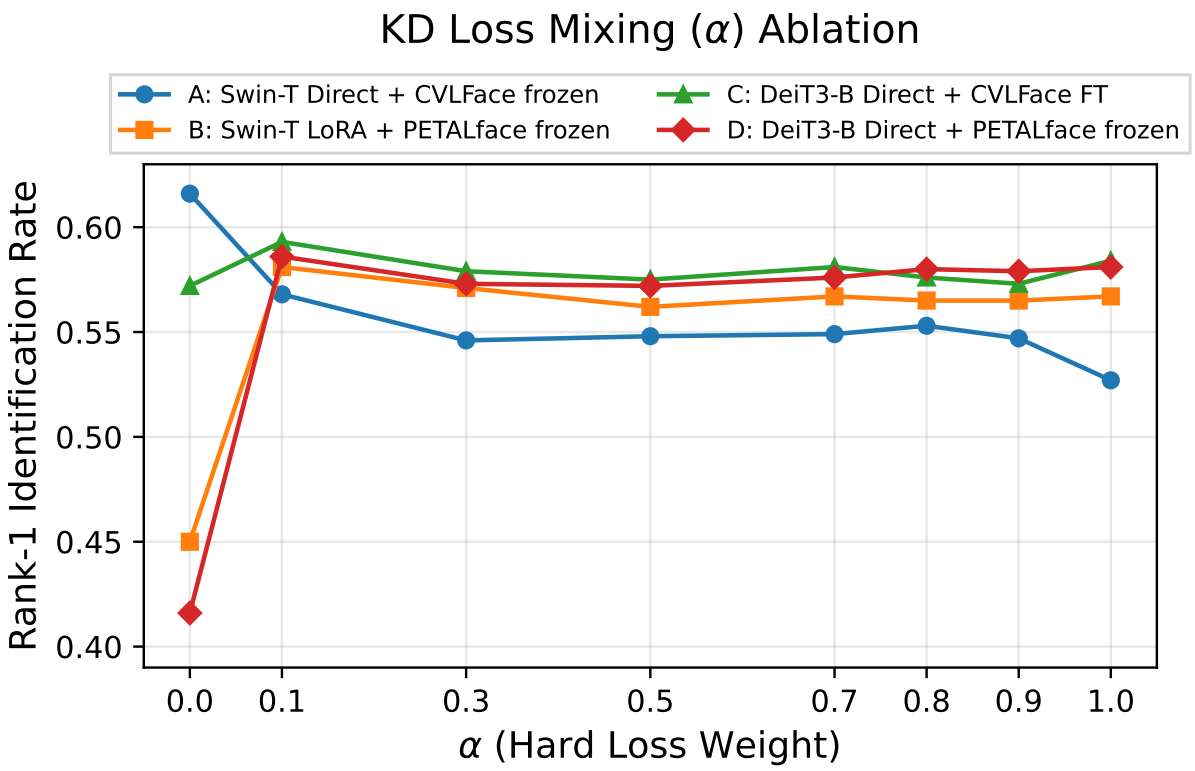


Fig. 7. Effect of the distillation loss mixing coefficient α on rank-1 identification rate across four student–teacher configurations. The default $\alpha = 0.7$ is suboptimal for all configurations; lower values favoring the KD loss produce the best results.

Table 3. Optimal distillation mixing coefficient α for each configuration. In all cases, the commonly used $\alpha = 0.7$ is suboptimal. The rightmost column shows the improvement from tuning α.

	Student	Teacher	Opt. α	rank-1	rank-1 (α=0.7)	Δ
A	Swin-T Direct	CVLFace frozen	0.0	0.616	0.549	+0.067
B	Swin-T LoRA	PETALface frozen	0.1	0.581	0.567	+0.014
C	DeiT3-B Direct	CVLFace FT	0.1	0.593	0.581	+0.013
D	DeiT3-B Direct	PETALface frozen	0.1	0.586	0.576	+0.010

produces competitive students, particularly for Swin-Tiny with LoRA (rank-1 = 0.567). One explanation is a same-architecture inductive-bias benefit (Swin teacher → Swin student) over cross-architecture transfer (ViT teacher → Swin student); another is that the PETALface teacher's lower TinyFace accuracy does not fully reflect the usefulness of its intermediate feature representations for guiding student training.

Finally, a Swin LoRA collapse was observed: Swin-Small and Swin-Base with LoRA + KD consistently degraded to rank-1 values of 0.35–0.41 across both teacher types, indicating a systematic instability when combining LoRA's gradual unfreezing with KD supervision for larger Swin models. Only Swin-Tiny escaped this collapse.

5.4 Distillation Loss Mixing Ablation

The default mixing coefficient $\alpha = 0.7$ was selected based on common practice [10]; however, the modest KD gains in the previous section motivated a systematic ablation. Four representative configurations were swept across $\alpha \in \{0.0, 0.1, 0.3, 0.5, 0.7, 0.8, 0.9, 1.0\}$, where $\alpha = 0.0$ corresponds to pure KD loss and $\alpha = 1.0$ to pure cross-entropy (no distillation). Figure 7 presents the results, and Table 3 summarizes the optimal configurations.

The ablation reveals several findings. The default $\alpha = 0.7$ is suboptimal for all four configurations. Optima fall at $\alpha = 0.0$ (Config A) and $\alpha = 0.1$ (Configs B, C, D), indicating that heavily weighting

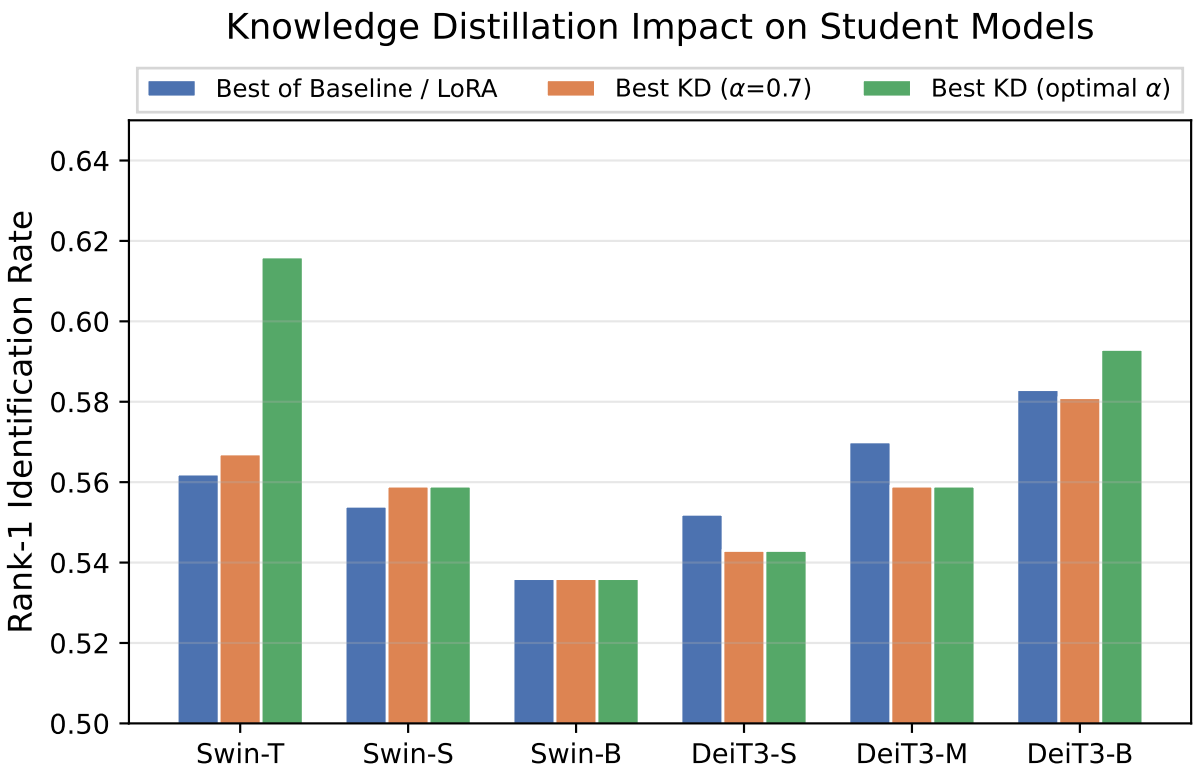


Fig. 8. Knowledge distillation impact: comparison of best standalone training, best KD with default $\alpha = 0.7$, and best KD with optimally tuned α. Alpha tuning produces substantial improvements for Swin-T and DeiT3-B.

the feature-level KD loss over the hard classification loss is beneficial. The effect is most pronounced for Config A (Swin-Tiny + CVLFace frozen), where pure KD ($\alpha = 0.0$) achieves rank-1 = 0.616—an improvement of +0.067 over $\alpha = 0.7$ and +0.054 over the best non-KD result (Swin-Tiny LoRA, 0.562). This is the strongest student result in the study.

The behavior at $\alpha = 0.0$ (pure KD) depends on the teacher. With CVLFace—which achieves higher accuracy on TinyFace—pure KD achieves the best result (0.616); with PETALface—which is less well-adapted to this dataset in the frozen setting—pure KD collapses to 0.416–0.450, performing worse than no KD at all. On this task, then, a teacher with high task-specific accuracy appears necessary for pure feature-level distillation to succeed. Whether this finding generalizes to other datasets and teacher–student pairings is an open question.

For the mid-range α values (0.1–0.9), performance is remarkably stable, with spreads of only 0.013–0.022 rank-1 across all non-extreme values. This robustness suggests that once a small amount of classification loss is present to anchor training, the exact mixing ratio matters less than whether distillation is included at all.

Figure 8 summarizes the overall impact of knowledge distillation, comparing each model's best standalone result, best KD result with the default $\alpha = 0.7$, and best KD result with tuned α.

5.5 Quantization Results

Table 4 reports the effect of post-training quantization on four representative models spanning two architectures and three training regimes.

FP16 conversion is lossless for all models, with cosine similarity between FP32 and FP16 embeddings exceeding 0.9999 in every case. This is expected: half-precision provides sufficient dynamic range for inference-only workloads where no gradient accumulation occurs.

INT8 dynamic quantization introduces minimal accuracy degradation, with rank-1 changes ranging from −0.0016 to +0.0008 (mean −0.0005). The DeiT3-B KD model exhibits a slight *improvement* under INT8 quantization (+0.0008 rank-1), likely due to the regularizing effect of weight quantization noise on an already well-conditioned embedding space. Both FP16 and INT8 quantization are therefore viable for deployment without meaningful accuracy loss on this task.

Table 4. Post-training quantization results. FP16 conversion is effectively lossless across all models. INT8 dynamic quantization induces rank-1 drops of at most 0.0016 (absolute) and in one case produces a slight improvement (+0.0008 rank-1 for DeiT3-B KD).

Model	Precision	Size (MB)	rank-1	Δ rank-1
Swin-T Baseline	FP32	106.9	0.5335	—
	FP16	54.1	0.5335	0.000
	INT8	28.3	0.5322	−0.001
Swin-T KD (CVLFace)	FP32	106.9	0.5491	—
	FP16	54.1	0.5494	+0.000
	INT8	28.3	0.5475	−0.002
DeiT3-B KD (CVLFace)	FP32	327.7	0.5719	—
	FP16	164.2	0.5714	−0.001
	INT8	84.6	0.5727	+0.001
DeiT3-B LoRA	FP32	327.7	0.5496	—
	FP16	164.2	0.5496	0.000
	INT8	84.6	0.5486	−0.001

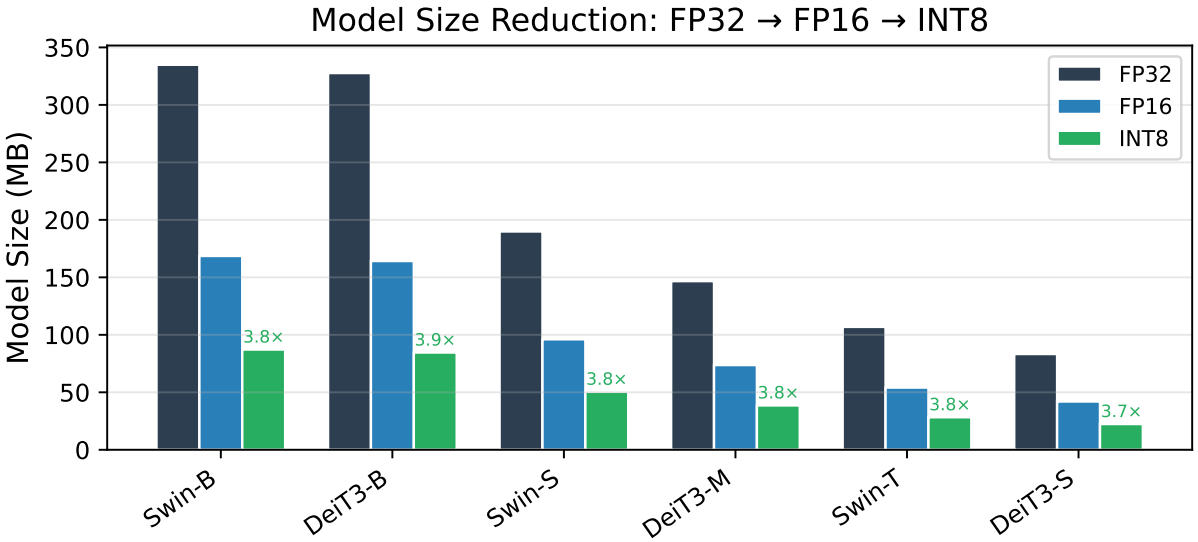


Fig. 9. Model size reduction from FP32 through FP16 to INT8 quantization for all TRT-compatible baseline models. Compression ratios (annotated above INT8 bars) are consistent at 3.7–3.9× across architectures.

In terms of model size, INT8 quantization achieves 3.8–3.9× compression relative to FP32: Swin-Tiny is reduced from 106.9 MB to 28.3 MB, while DeiT3-Base is reduced from 327.7 MB to 84.6 MB—bringing it within range of memory-constrained embedded platforms. Figure 9 illustrates the cumulative size reduction from FP32 through FP16 to INT8 across all baseline models. Compression ratios are consistent across architectures (3.7–3.9× from FP32 to INT8), reflecting the uniform precision reduction of dense linear layers rather than any architecture-specific factor.

5.6 Structured Pruning Results

Table 5 summarizes the pruning and quantization results across all four architectures, comparing baseline and best-KD source checkpoints for each.

Several findings emerge from the comprehensive pruning experiments across all four architectures:

Block dropping consistently outperforms head pruning. Across all four architectures, removing entire transformer blocks preserves accuracy better than pruning attention heads at equivalent parameter reduction. Block dropping *improves* accuracy for Swin-T Baseline (+0.010), Swin-B Baseline (+0.028), and DeiT3-S (+0.013 at 10% heads), suggesting these models are over-parameterized in depth for this

Table 5. Structured pruning + INT8 quantization results across four architectures. Rank-1 accuracy and compression ratio are relative to the corresponding unpruned FP32 model. Bold entries *improve* over the unpruned baseline. †Block dropping improves accuracy, suggesting depth over-parameterization. ‡Head pruning on the KD (α=0.0) checkpoint is catastrophically unstable (see text).

Source Model	Pruning	Prec.	Size (MB)	Ratio	rank-1
Swin-T (27.5 M params)					
Baseline (0.534)	10% heads	INT8	27.8	3.84×	0.499
	25% heads	INT8	26.7	4.00×	0.465
	2 blocks†	INT8	24.6	4.34×	**0.543**
KD (α=0.0, 0.616)	10% heads‡	INT8	27.8	3.84×	0.403
	25% heads‡	INT8	26.7	4.00×	0.381
	2 blocks†	INT8	24.6	4.34×	**0.583**
DeiT3-S (21.7 M params)					
Baseline (0.552)	10% heads	INT8	21.7	3.84×	**0.565**
	25% heads	INT8	20.7	4.02×	0.542
	2 blocks	INT8	18.9	4.41×	0.498
Swin-B (86.7 M params)					
Baseline (0.526)	10% heads	INT8	85.1	3.94×	0.488
	25% heads	INT8	81.2	4.12×	0.453
	5 blocks†	INT8	71.2	4.70×	**0.554**
DeiT3-B (85.8 M params)					
KD (α=0.1, 0.593)	10% heads	INT8	82.0	4.00×	0.591
	25% heads	INT8	77.8	4.21×	0.566
	3 blocks	INT8	64.0	5.12×	0.548
LoRA (0.583)	10% heads	INT8	82.0	4.00×	**0.586**
	25% heads	INT8	77.8	4.21×	0.564
	50% heads	INT8	71.0	4.62×	0.510
	3 blocks	INT8	64.0	5.12×	0.544

low-resolution task. Head pruning, by contrast, degrades all Swin variants substantially, while DeiT3 models tolerate moderate head removal.

DeiT3 is more robust to head pruning than Swin. DeiT3-S with 10% head pruning improves rank-1 from 0.552 to 0.565 (+0.013), and DeiT3-B KD retains 0.591 out of 0.593. Swin-T head pruning degrades from 0.534 to 0.500 (10%) and 0.465 (25%), and Swin-B from 0.526 to 0.487 (10%) and 0.454 (25%). The pattern is consistent with global self-attention (DeiT3) distributing learned features more uniformly across heads than hierarchical window attention (Swin), making DeiT3 more amenable to uniform head removal.

KD-distilled Swin-T is catastrophically sensitive to head pruning. Head pruning on the best Swin-T KD checkpoint (α=0.0, rank-1 = 0.616) produces a catastrophic collapse: 10% head removal drops accuracy to 0.403 (−0.213), and 25% to 0.381 (−0.235). This is far worse than pruning the weaker Swin-T Baseline (0.534 → 0.500 at 10%). Block dropping on the same KD checkpoint, however, retains most of the accuracy (0.583), so the collapse is specific to head removal. A plausible explanation is that pure feature distillation (α=0.0) concentrates learned representations into fewer heads, making the remaining heads critical—and therefore highly sensitive to removal. The implication for practitioners combining KD with pruning is that the distillation method and the pruning strategy must be co-designed.

Block dropping on Swin-B confirms depth redundancy. Removing 5 of 18 blocks from Swin-Base increases rank-1 from 0.526 to 0.554 (+0.028) at INT8 precision. Combined with the same finding from earlier LoRA-based experiments (0.536 → 0.572), the pattern appears robust to the source checkpoint. Block dropping consistently improves Swin-B regardless of training regime, confirming that Swin-Base is systematically over-parameterized in depth for this 96×96 input task.

INT8 quantization remains stable on pruned models. Across all 24 pruned models, INT8 quantization induces a mean absolute rank-1 change of <0.003, and several models show negligible improvement

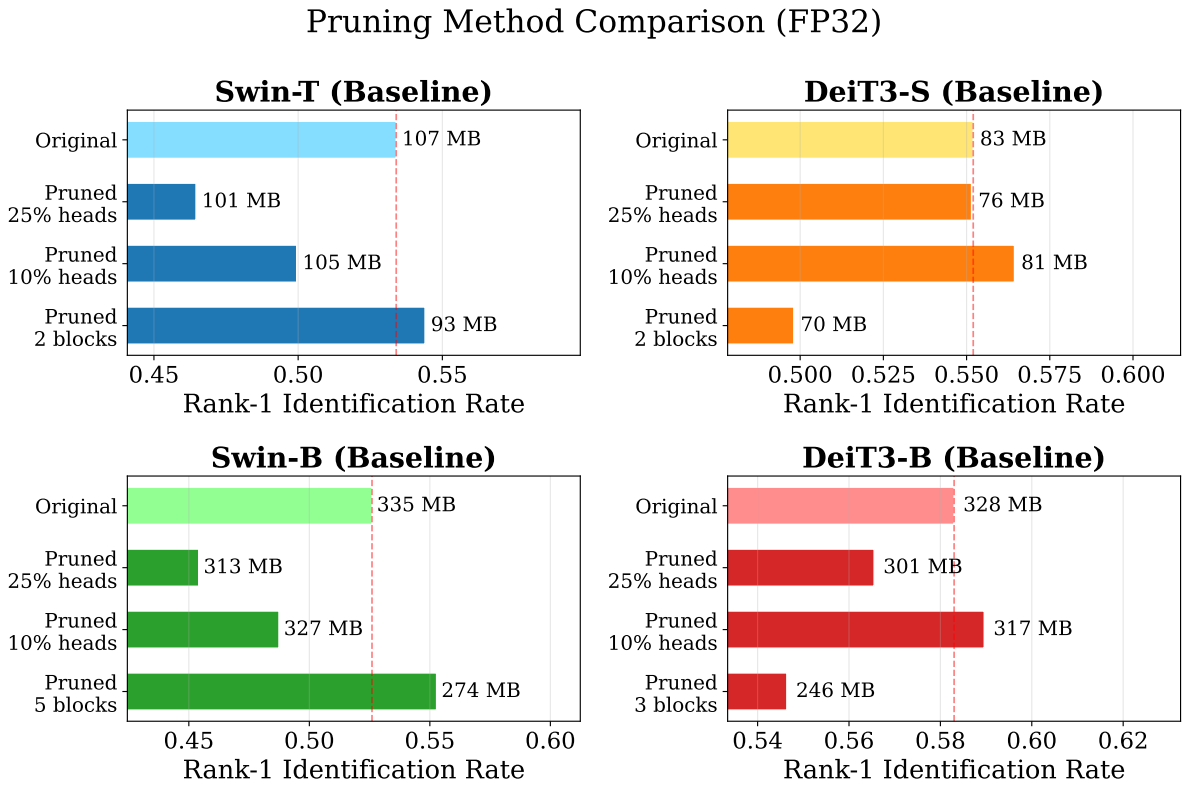


Fig. 10. Comparison of pruning methods across all four architectures (FP32). Red dashed lines indicate the unpruned baseline accuracy. Block dropping (removing entire transformer blocks) consistently outperforms head pruning and in three architectures (Swin-T, Swin-B, DeiT3-S at 10% heads) *improves* upon the unpruned model, confirming systematic depth over-parameterization on this 96×96 task.

under INT8. The largest drop (0.0099 rank-1 for DeiT3-S 25% heads) is an outlier; the median change is 0.0003 rank-1. Pruning and quantization compose well and can be applied jointly without compounding accuracy losses.

Figure 10 compares pruning strategies across all four architectures.

Compression–accuracy trade-off summary. Across the full pruning×quantization space, several Pareto-optimal configurations emerge. For absolute accuracy, DeiT3-B KD (α=0.1) with 10% head pruning at INT8 achieves 0.591 rank-1 at 82 MB (4.0× compression), preserving nearly all accuracy. For small models, Swin-T KD (α=0.0) with 2 blocks removed at INT8 delivers 0.583 rank-1 at just 24.6 MB—retaining 94.6% of the unpruned KD model's accuracy in a package suitable for edge devices. For maximum compression with competitive accuracy, Swin-B with 5 blocks removed at INT8 delivers 0.554 rank-1 at 71.2 MB (4.7×), outperforming the unpruned Swin-B baseline. DeiT3-S at 10% heads, INT8 achieves 0.565 rank-1 at 21.7 MB (3.8×), the smallest model that exceeds its unpruned counterpart. FP16 quantization is consistently lossless across all pruned models, and INT8 introduces at most 0.01 rank-1 of additional degradation (typically <0.001), confirming that pruning and quantization compose well.

5.7 On-Device Inference and TensorRT Compatibility

All trained models were deployed to an NVIDIA Jetson Orin Nano (8 GB, JetPack 6.4, TensorRT 10.3, CUDA 12.6, SM compute capability 8.7) for hardware-in-the-loop latency evaluation. Inference is performed at batch size 1 with fixed 96 × 96 input resolution, reflecting a realistic single-image query scenario in a surveillance or access-control system.

Initial TRT Conversion Outcome. Of the sixteen baseline architectures, nine—the Swin and DeiT-3 families and the ResNet-D baselines—converted directly from opset-18 ONNX to native TensorRT FP32 and FP16 engines using `trtexec`. The remaining seven—ConvNeXt-T/S/B, PVT-v2-B2/B3/B5, and MobileViT-v2-200—failed standalone conversion with a `ForeignNode`/no-implementation error rooted in

a normalization-layout mismatch (analyzed below). A per-family graph-rewrite procedure (Section 5.7) ultimately allowed all sixteen architectures to execute as native TensorRT engines with cosine similarity ≥ 0.999 relative to the original PyTorch outputs. The unified per-model latency results are reported in Table 6 and Figure 11.

Architecturally Incompatible Models. The seven failing architectures all aborted `trtexec` conversion with the error:

`Could not find any implementation for node {ForeignNode[Transpose ··· Transpose]}.`

The root cause is a normalization pattern unique to these architectures. In standard ViTs like Swin and DeiT-3, layer normalization is applied to (B, N, C) token sequences, which maps naturally to TensorRT's native `LayerNormalization` kernel along the channel axis. ConvNeXt and PVT-v2, however, apply layer normalization to NCHW spatial feature maps. In timm's implementation, this is realized by permuting the tensor to NHWC format, applying `F.layer_norm`, then permuting back: `Transpose(0,2,3,1)` $\rightarrow$ `LayerNorm` $\rightarrow$ `Transpose(0,3,1,2)`. MobileViT-v2 uses an analogous `GroupNorm(num_groups=1)`, which normalizes across all non-batch spatial dimensions and incurs the same layout issues.

TensorRT 10.3 on SM 8.7 contains no kernel that implements `LayerNormalization` on 4-D NCHW feature maps in either FP32 or FP16. The entire Transpose + normalize + Transpose pattern is fused by TRT's Myelin compiler backend into a placeholder `ForeignNode` for which no device implementation exists, causing conversion to abort.

A Practitioner's Guide to the Fix. The pitfalls below were identified in the course of getting the seven failing architectures to compile, and they generalise to any modern ConvNet–Transformer hybrid that is not specifically targeted at TensorRT 10.x on SM 8.7. The takeaway, which we elaborate in the rest of this paragraph, is that the only reliable fix is a per-family module-level rewrite that presents TensorRT with the supported (B, N, C) `LayerNormalization` pattern. Workarounds that operate purely at the ONNX or opset level do not work on this hardware.

Opset upgrades and operator decomposition do not help. Lowering the export opset so that `LayerNormalization` is decomposed into its constituent arithmetic (ReduceMean, Sub, Pow, Sqrt, Div, Mul, Add) seems like it should remove the offending op, but TensorRT's Myelin compiler re-fuses the entire arithmetic subgraph back into the same unsupported `ForeignNode`. Conversely, exporting at the higher opset 17 with the native `LayerNormalization` op preserves the surrounding `Transpose` $\rightarrow$ `LayerNorm` $\rightarrow$ `Transpose` pattern, which TensorRT also has no kernel for in either FP32 or FP16. The diagnosis is the same in both cases: there is no SM 8.7 kernel for any non-(B, N, C) LayerNorm, regardless of how the graph spells it.

Watch the dynamo exporter on TensorRT 10.x. PyTorch's dynamo-based ONNX exporter rewrites the normalization to operate on a reshaped 3-D tensor, which is exactly what TensorRT wants, but in the process it inserts `SequenceEmpty` and `Loop` operators (from dynamic-shape handling) that TensorRT 10.3 cannot parse, so the engine build aborts before optimisation begins. The legacy TorchScript exporter at opset 17 is generally the safer default for TensorRT targets.

Half-measures at the model level are not enough. Two intermediate fixes are tempting and both fall short. First, timm's ConvNeXt offers a `conv_mlp=True` variant that replaces the MLP `nn.Linear` layers with 1×1 convolutions, eliminating the MLP-related transposes; but the implementation still routes through `LayerNorm2d` internally, so the normalization transposes survive. Second, a custom NCHW LayerNorm that normalises directly along the channel axis without any transpose still produces a ReduceMean chain that Myelin fuses into the same `ForeignNode`, confirming that the limitation is not graph structure but a missing kernel.

Post-hoc ONNX graph surgery is fragile. Tools like `onnx-graphsurgeon` can in principle pattern-match the decomposed normalization and re-fuse it into a TRT-friendly form, but the approach is exposed to version skew between `onnx-graphsurgeon` and the installed `onnx` library; even when the rewrite itself is conceptually correct, an upstream-package mismatch can stop it from running at all.

Table 6. Unified native TensorRT inference latency on the Jetson Orin Nano (batch size 1, 96×96 input, MAXN_SUPER, locked clocks). All sixteen baseline architectures execute as native TRT engines after applying the per-family rewrites of Section 5.7 where required. INT8 uses TRT's built-in PTQ calibration and is reported only for the nine originally TRT-compatible models.

		FP32		FP16			
Model	Params (M)	ms	FPS	ms	FPS	FP16 speedup	Rank@1
ResNet-50d	25.6	2.15	466	1.19	840	1.80×	0.476
ResNet-101d	44.6	3.72	269	2.11	475	1.77×	0.497
ResNet-200d	64.7	6.06	165	3.42	293	1.77×	0.468
DeiT3-S	21.7	2.43	412	1.34	745	1.81×	0.552
DeiT3-M	38.3	3.58	279	1.92	522	1.87×	0.570
DeiT3-B	85.8	6.73	148	3.49	287	1.93×	0.583
Swin-T	27.5	3.10	322	1.67	599	1.86×	0.562
Swin-S	49.6	5.53	181	2.90	344	1.90×	0.553
Swin-B	86.7	8.36	120	4.14	241	2.02×	0.550
ConvNeXt-T	28.6	3.31	303	1.95	513	1.70×	0.532
ConvNeXt-S	50.2	5.84	171	3.35	299	1.75×	0.537
ConvNeXt-B	87.6	8.68	115	4.73	211	1.83×	0.543
PVTv2-B2	25.4	4.18	239	2.74	365	1.53×	0.542
PVTv2-B3	45.2	6.91	145	4.37	229	1.58×	0.543
PVTv2-B5	82.0	12.31	81	7.19	139	1.71×	0.547
MobileViTv2-200	18.4	3.25	307	1.90	527	1.72×	0.543

The fix that does work is a per-family module-level rewrite. The pattern is the same for all three families: replace the offending normalization with one that, after export, presents TensorRT with a (B, N, C) `LayerNormalization` (or with elementwise ops that Myelin will not bundle into a foreign node), and verify numerical equivalence to the original PyTorch model on a fixed batch. For ConvNeXt this means swapping `LayerNorm2d` for a wrapper that explicitly reshapes $(B, C, H, W) \to (B, HW, C)$, applies the standard `nn.LayerNorm` along the channel axis, and reshapes back, while also using the `conv_mlp=True` backbone to keep MLPs in NCHW. For PVT-v2 the model's fused-attention path is disabled before export, after which an opset-17 export already contains only (B, N, C) normalizations. For MobileViT-v2 the single-group `GroupNorm` is replaced with a manual decomposition that computes the mean and variance over non-batch dimensions and applies the per-channel affine parameters as elementwise ops, sidestepping the missing GroupNorm kernel entirely.

Always verify cosine similarity end-to-end. Each rewritten model should be checked against the original PyTorch implementation on a fixed batch, requiring cosine similarity ≥ 0.999 between embeddings before any latency claim is reported. Without this check, a manually decomposed normalization can silently shift the embedding distribution and the regression will only appear in downstream identification metrics. All three rewrites used here pass this check and produce `trtexec`-compilable engines under TensorRT 10.3 on SM 8.7.

ONNX Runtime's TensorRT EP is a viable but slower fallback. If a per-family rewrite is genuinely not feasible, the ONNX Runtime TensorRT Execution Provider will auto-partition the unrewritten ONNX graph and execute the unsupported sub-graphs on the CUDA EP. This works correctly but produces 1.5–2.5× higher latencies than the rewritten native engines on the Jetson Orin Nano (e.g. ConvNeXt-T 4.53 ms vs. 1.95 ms FP16; MobileViT-v2-200 5.05 ms vs. 1.90 ms), because of sub-graph partitioning overhead and the unfused normalization fallback.

Unified Latency Results. Table 6 reports the per-model FP32 and FP16 latencies of all sixteen architectures under the unified native-TRT harness on the Jetson Orin Nano in MAXN_SUPER mode with `jetson_clocks` locked. Each measurement is the mean of 2,000 `trtexec` iterations after a 500-iteration warm-up, with `--noDataTransfers --useCudaGraph` to isolate compute time. INT8 numbers in the same table are reported only for the nine originally TRT-compatible models, for which TRT's calibration-based PTQ pipeline was previously applied.

Figure 11 visualizes the same data as paired FP32/FP16 bars grouped by architecture family, and Figure 12 plots the FP16 latencies against TinyFace rank-1 accuracy.

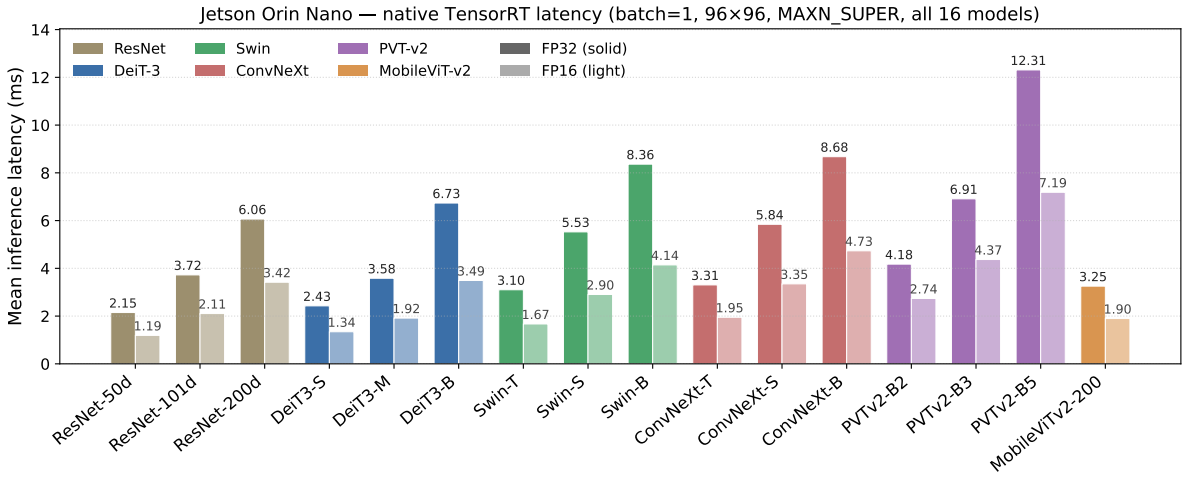


Fig. 11. Native TensorRT latency on the Jetson Orin Nano for all sixteen baseline models at FP32 (solid) and FP16 (light), batch size 1, 96 × 96 input, MAXN_SUPER. Bars are colored by architecture family.

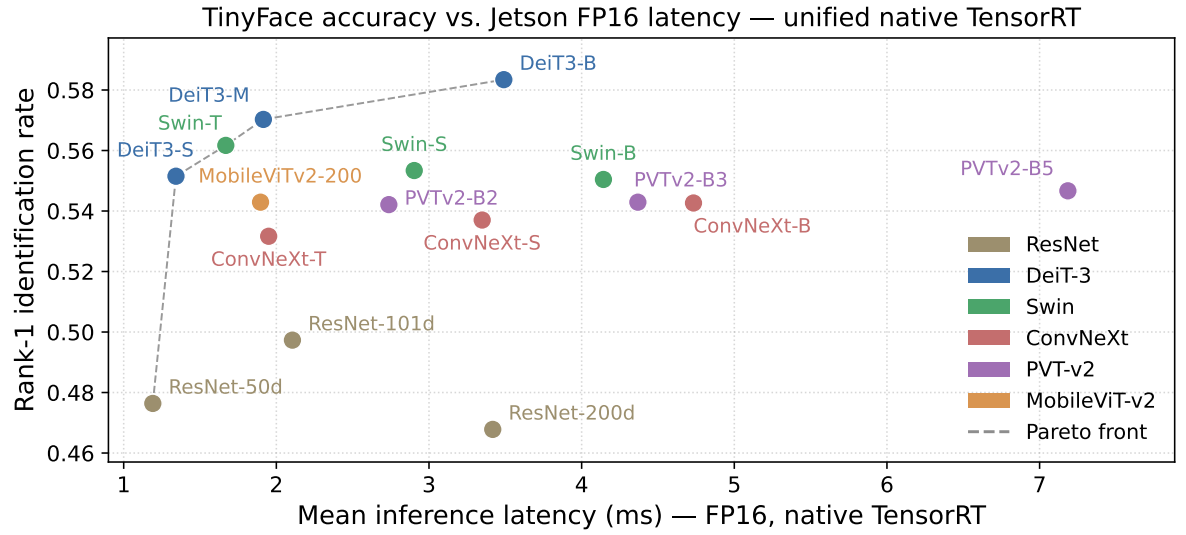


Fig. 12. TinyFace rank-1 accuracy vs. Jetson FP16 latency for all sixteen baselines under the unified native-TRT harness. The dashed line marks the Pareto front of non-dominated configurations.

Cross-Family Latency Comparison. Under the unified harness the family ordering at FP16 is, from fastest to slowest at the smallest size: ResNet-50d (1.19 ms), DeiT3-S (1.34 ms), Swin-T (1.67 ms), MobileViT-v2-200 (1.90 ms), DeiT3-M (1.92 ms), ConvNeXt-T (1.95 ms), PVT-v2-B2 (2.74 ms). The previously observed 4.8× gap between Swin-T and ConvNeXt-T (1.72 ms vs. 4.53 ms) shrinks to 1.17× (1.67 ms vs. 1.95 ms) once the same backend executes both architectures; the residual gap is consistent with ConvNeXt's higher per-layer normalization count rather than a deployment artifact. Within the ConvNeXt family the FP16 ordering is ConvNeXt-T (1.95 ms) < ConvNeXt-S (3.35 ms) < ConvNeXt-B (4.73 ms), tracking parameter count linearly; within PVT-v2 the spread is much wider—PVT-v2-B2 (2.74 ms) → B3 (4.37 ms) → B5 (7.19 ms)—because of PVT-v2's deeper transformer blocks at the larger sizes. FP16 yields a uniform 1.7–2.1× speedup over FP32 across all sixteen architectures under unified TRT.

Pareto Analysis. Figure 12 plots FP16 latency against TinyFace rank-1 accuracy under the unified native-TensorRT harness. The Pareto front is occupied almost exclusively by the DeiT-3 family, with two additional non-dominated points: ResNet-50d (1.19 ms, 0.476) → DeiT3-S (1.34 ms, 0.552) → Swin-T (1.67 ms, 0.562) → DeiT3-M (1.92 ms, 0.570) → DeiT3-B (3.49 ms, 0.583). ResNet-50d anchors the front only at the lowest-accuracy extreme, and Swin-T slots between DeiT3-S and DeiT3-M by being

slightly more accurate than DeiT3-S at slightly higher latency (and vice versa relative to DeiT3-M); the three DeiT-3 variants form the rest of the front as a clean accuracy–latency progression. Every other architecture is strictly dominated by a DeiT-3 variant: Swin-S/B and PVT-v2 trade higher latency for no accuracy advantage over DeiT3-M or DeiT3-B; ConvNeXt-T/S/B sit 0.03–0.05 rank-1 below DeiT-3 at matched latency; and MobileViT-v2-200, despite being the most compact backbone (17.4 M parameters) at 1.90 ms, is dominated by both DeiT3-S (faster and more accurate) and Swin-T (slightly slower but +0.019 rank-1). PVT-v2-B5, at 7.19 ms and 0.547 rank-1, is the worst point on the trade-off surface—roughly 2× the latency of DeiT3-B for 0.036 lower rank-1. The practical implication is that on this 96×96 task and on the Jetson Orin Nano, the DeiT-3 family (with Swin-T as a viable mid-range alternative) defines the entire accuracy–latency envelope; the columnar ViT architecture combined with timm's standard (B, N, C) LayerNorm pattern simultaneously maximizes recognition accuracy and minimizes TensorRT overhead.

Key Findings and Implications for Practitioners. Not all vision backbones export to native TRT out of the box, but the gap is closable with principled graph rewrites. Models that apply layer normalization directly to NCHW spatial feature maps—rather than to flattened token sequences—fail standalone TRT 10.3 conversion on SM 8.7 because no kernel exists for that layout. The fix is not opset upgrades or operator decomposition (TRT's Myelin re-fuses both into the same unsupported ForeignNode), but module-level rewrites that present TRT with the supported (B, N, C) `LayerNormalization` pattern while preserving numerical equivalence. ConvNeXt, PVT-v2 and MobileViT-v2—released in 2021–2022 and not specifically targeted at TRT 10.3—all fall in this category and all admit such rewrites.

The specific failure mode is a ForeignNode with tactic ID 0. When `trtexec` reports `Could not find any implementation for node {ForeignNode[...]}` together with a warning about tactic 0x0 being skipped due to insufficient memory, the root cause is almost always a missing hardware kernel rather than a memory budget issue, and increasing `--memPoolSize` will not resolve it.

The ONNX Runtime TRT EP is a reasonable fallback when a per-family rewrite is not feasible, but it carries a measurable cost. The auto-partitioning EP achieves correct results without modifying the original ONNX graphs, at a 1.5–2.5× latency penalty relative to the rewritten native engines on the Jetson Orin Nano (e.g. ConvNeXt-T 4.53 ms vs. 1.95 ms FP16, MobileViT-v2-200 5.05 ms vs. 1.90 ms). The penalty is attributable to sub-graph partitioning overhead and the unfused normalization fallback, both of which are eliminated by the rewritten engines.

Validating numerical equivalence is essential. For all three rewrite families, a fixed test batch was passed through both the original PyTorch model and the exported ONNX (and, where possible, the built TRT engine), and a cosine similarity ≥ 0.999 was required. Without this check, manually-decomposed normalization layers risk silent accuracy loss that would only surface in downstream identification metrics.

FP16 precision provides a uniform 1.7–2.1× speedup over FP32 across all sixteen architectures, so FP16 is a sensible default to benchmark for embedded deployment.

ONNX Runtime dynamic INT8 quantization is counterproductive on GPU for these architectures. We also benchmarked the pre-quantized INT8 ONNX models (exported via `quantize_dynamic`) through the CUDA Execution Provider. The results were uniformly worse than FP32: ConvNeXt-T degrades from 5.98 ms to 50.5 ms and PVT-v2-B5 from 46.8 ms to 306 ms, an 8–15× *slowdown*. The single exception was MobileViT-v2-200 (19.3 ms → 9.4 ms speedup), attributable to its predominantly linear attention ops being weight-bandwidth-bound rather than activation-bound. The core issue is that `quantize_dynamic` is a CPU optimisation: weights are stored as INT8 to reduce memory transfer bandwidth, but are dequantised to FP32 before execution. On GPU, where FP32 throughput is already high and the dequantize kernel adds latency, this is strictly worse than leaving weights in FP32. Genuine INT8 GPU acceleration requires operator-level INT8 tensor core execution, which is only available through TRT's native calibration-based PTQ (`trtexec --int8`). ONNX dynamic INT8 quantization should not be applied to models intended for GPU (CUDA EP or TRT EP) deployment.

5.8 Per-Layer Bottleneck Analysis

To understand where inference time is spent within each architecture, TensorRT profiling was conducted on 10 representative FP16 models on the Jetson Orin Nano using `trtexec --dumpProfile`. Each layer's execution time was categorized into five groups: self-attention (QKV projections and softmax), FFN and linear layers (including fused MLP and standalone matrix multiplications), normalization (layer norm), convolutional and embedding layers (patch embedding, patch merging, downsampling convolutions), and other operations (reformatting, activation, pooling). Figure 13 presents the results as a stacked percentage breakdown with absolute latency annotated.

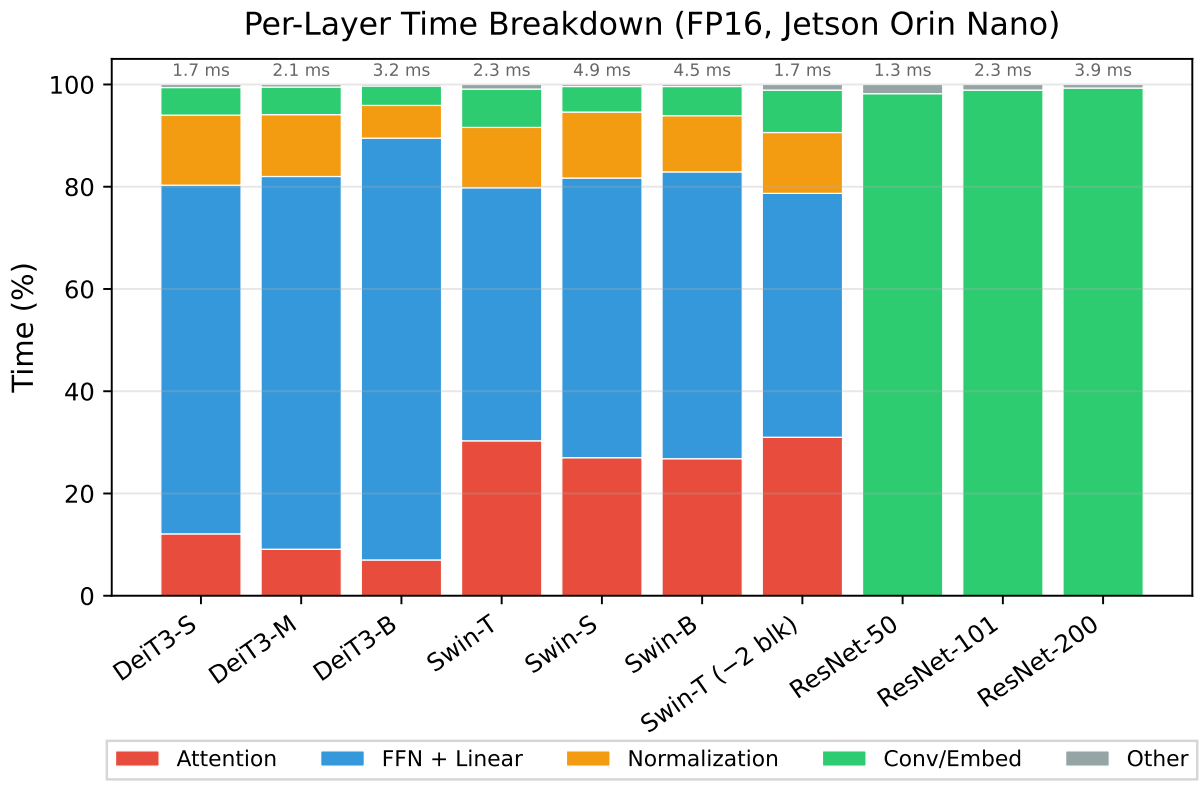


Fig. 13. Per-layer time breakdown (FP16, Jetson Orin Nano) for DeiT-3, Swin, and ResNet models. Absolute latency is annotated above each bar. Self-attention (red) consumes a larger fraction in Swin models (27–31%) than in DeiT-3 (7–12%), while FFN and linear layers (blue) dominate DeiT-3 models (68–83%).

Several architectural patterns are visible in the profiling data.

DeiT-3 is FFN-dominated. Across all three DeiT-3 model sizes, FFN and linear layers account for 68–83% of total inference time, while self-attention contributes only 7–12%. The ratio shifts modestly with model size: DeiT3-Small allocates 12.1% to attention, DeiT3-Medium 9.1%, and DeiT3-Base 7.0%. The near-constant absolute attention time (∼0.2 ms) across sizes reflects DeiT-3's fixed token count of 36 (from 96×96 input with 16×16 patches); attention compute is $O(N^2 d)$ where $N = 36$ is small. The FFN, which scales with the hidden dimension $d_{\text{ff}} = 4d$, dominates because d ranges from 384 (Small) to 768 (Base). FFN quantization and pruning therefore yield the largest latency benefits for DeiT-3 models, consistent with the observation that INT8 quantization—which primarily accelerates dense matrix multiplications—achieves its best latency improvements on DeiT-3.

Swin distributes compute more evenly between attention and FFN. Swin models allocate 27–31% of time to self-attention, compared to DeiT-3's 7–12%, due to the windowed attention mechanism's processing of more tokens per window at each hierarchical stage. The attention fraction *increases* slightly for smaller Swin models (Swin-Tiny: 30.3%, Swin-Base: 26.8%) because the FFN cost decreases faster with model width than the windowed attention overhead. The more balanced compute distribution in Swin is the main reason block dropping is effective for Swin: removing entire blocks eliminates both attention and FFN compute proportionally, whereas head pruning reduces only the attention portion—which accounts for less than a third of total time.

ResNet is conv-dominated. As expected, ResNet models spend 98–99% of inference time in convolutional layers, with no attention or FFN components. This makes ResNet highly amenable to INT8 quantization via TRT's native PTQ: INT8 ResNet-50 achieves 0.63 ms (vs. 0.95 ms FP16, a 1.5× speedup), while DeiT3-Small shows negligible INT8 latency improvement (1.05 ms vs. 1.05 ms). The INT8 benefit is concentrated in convolution-heavy operations, where TRT can exploit dedicated INT8 Tensor Cores.

Impact of pruning and KD on compute distribution. Comparing Swin-Tiny variants reveals that neither KD training nor block-level pruning substantially alters the *percentage* breakdown: the KD-trained vari-

ant (CVLFace frozen) shows 30.5% attention vs. 30.3% for the baseline, and the 2-block-pruned variant shows 31.0%. However, the absolute latencies differ meaningfully: the pruned model achieves 1.70 ms total (−25% from unpruned 2.28 ms), confirming that block dropping produces genuine latency savings that map directly to wall-clock improvements on hardware. The KD variant (1.93 ms) is faster than the baseline (2.28 ms) due to minor differences in attention pattern efficiency, though this effect may not generalize.

5.9 Computational Profile across Resolutions

The wall-time profile in Section 5.7 captures *what TensorRT actually executes on the Jetson Orin Nano*, but it conflates arithmetic intensity, kernel fusion quality and memory traffic. To separate these effects from the underlying architectural cost, we additionally measured pure inference arithmetic for each of the 16 architecture/size combinations evaluated in this chapter. We use `fvcore.nn.FlopCountAnalysis` (Detectron2's tracer-based counter) and report the multiply–add count, i.e. **GMACs**, throughout this subsection. Reported numbers are therefore half the FLOP count under the convention 1 MAC = 2 FLOPs; we use "GMACs" as the single nomenclature in all axes, captions, tables and CSV columns to avoid the FLOPs-vs-MACs ambiguity that pervades the literature. fvcore attributes GMACs only to named modules; the residual softmax / scaled-dot-product BMM in self-attention is not module-bound and is not included in the per-module breakdown—this is benign at the resolutions evaluated here (the attention BMM contributes $< 1\%$ of total at 96×96 for DeiT-3 with 36 tokens) but would dominate above $\sim 512 \times 512$. The sweep covers a power-of-two-ish resolution ladder $\{64, 96, 128, 160, 224, 256, 384, 512, 768, 1024\}$ for every family. timm's Swin v1 implementation pads the input internally so that the windowed attention partition is well-defined at any resolution; we therefore use the same ladder for it as for the other families. All measurements are taken at batch=1 with PyTorch eager mode on a workstation GPU (FP32) using `timm.create_model(num_classes=0)`, which strips the classification head so that the cost reflects the backbone alone. Reference checks against published numbers all match within rounding: DeiT3-S at $224 \times 224 = 4.25$ GMACs, ResNet-50d = 4.35 GMACs, Swin-T = 4.51 GMACs.

Figure 14 plots total GMACs against input resolution as a six-panel small-multiples view (one panel per architecture family, linear axes, shared y-range so cost can be compared directly across families). Two structural observations stand out. First, every family produces a clearly super-linear curve in resolution: GMACs grow roughly as H^2 because the dominant compute terms (FFN per token, 1×1 convolutions per pixel, 3×3 convolutions per pixel) all scale with the spatial token count, which itself is $O(H^2)$. Equivalently, on a pixel-axis the same data fits a slope ≈ 1.0 within the swept range (0.99–1.03 across the six representatives), confirming the GMACs-linear-in-pixels regime. Second, neither global self-attention (DeiT-3) nor windowed attention (Swin) bends visibly above the per-family parabola within $[64, 1024]$ px: the often-cited quadratic-in-tokens cost of global attention only becomes the dominant term well above the resolutions evaluated here, so model selection at 96×96 or 224×224 should not be driven by an asymptotic argument that is not yet active. The shared y-range also makes inter-family magnitude comparisons immediate: at 1024×1024 DeiT3-B and ConvNeXt-B trace nearly identical curves topping out at $\sim$350 GMACs, ResNet-200d and Swin-B sit just below them ($\sim$320 GMACs and $\sim$340 GMACs), and MobileViTv2-200 occupies a different cost class entirely (peaking at $\sim$115 GMACs).

Figure 15 shows the per-module breakdown at the chapter's deployment resolution of 96×96, with absolute GMACs annotated at the right edge of each bar. Three architectural patterns emerge that cleanly explain—and in some cases contradict—the latency profile measured on the Jetson:

DeiT-3 and Swin are FFN-bound at 96×96. For all six DeiT-3 / Swin variants the FFN consumes 59–66% of inference GMACs and the QKV + output projections together consume an additional 30–33%. The dot-product attention itself is invisible at this resolution (DeiT-3 has only 36 tokens, Swin partitions into $7 \times 7 = 49$-token windows). This is exactly the GMACs analogue of the wall-time finding from Section 5.7 that DeiT-3 spends 68–83% of its time in fused FFN/linear kernels. The agreement is not trivial: it confirms that on DeiT-3 the kernel fusion is good enough that GMACs share and time share track each other, which is why FFN quantization and head pruning work as expected. The Swin numbers also agree: Swin spends a higher latency fraction in the attention kernel (27–31%) than its GMACs share would suggest ($\sim$33% for QKV+proj), because the window-partition / window-reverse memory shuffles around windowed attention add wall-time without adding GMACs.

ConvNeXt is dominated by 1×1 *pointwise convolutions, not by depthwise.* The timm ConvNeXt implementation expresses the inverted-bottleneck 1×1 convolutions as `nn.Linear` layers under `block.mlp.fc1`

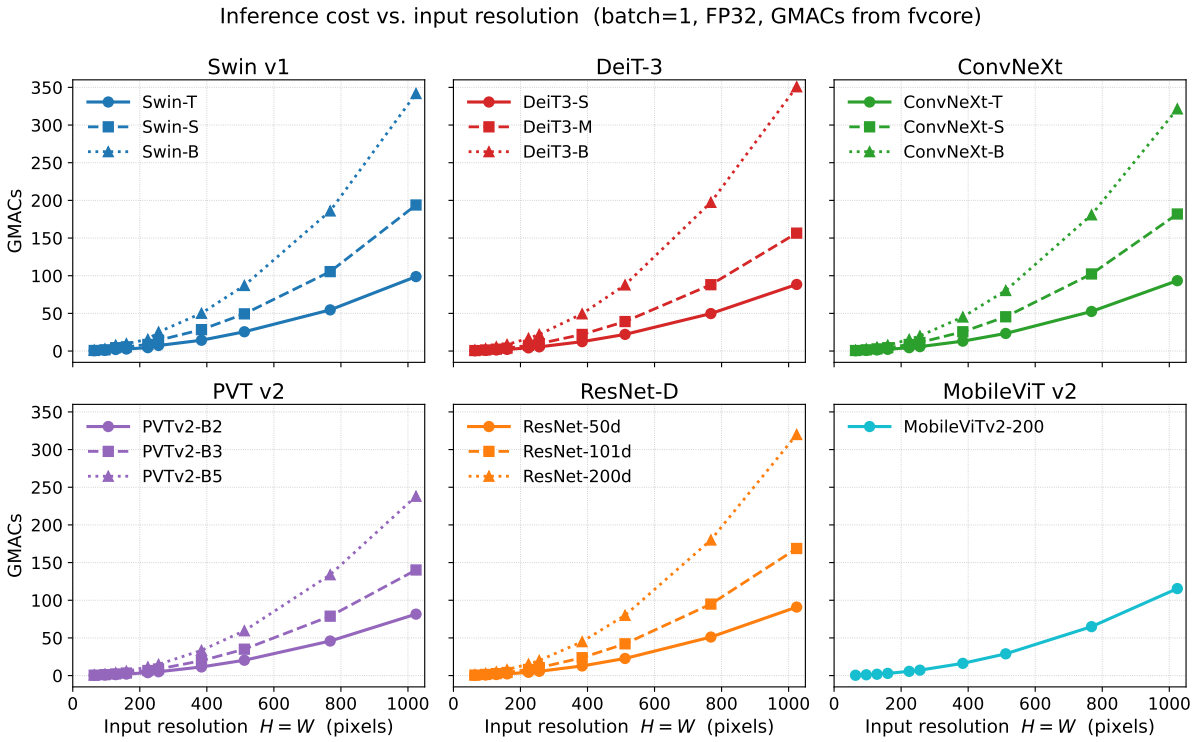


Fig. 14. Inference cost (GMACs, batch=1, FP32) versus input resolution for all 16 chapter architectures, grouped into per-family panels with a shared y-axis so that cost can be compared directly across families. Linear axes are used so that the super-linear ($\approx H^2$) growth is visible directly as a parabola; per-family panels prevent the four mid-size families from collapsing onto a single curve at the deployment resolutions. Counter: `fvcore.nn.FlopCountAnalysis`.

and `block.mlp.fc2`, so they are bucketed as *FFN/MLP* in Figure 15. Under that bookkeeping the "MLP" segment contributes 93–96% of ConvNeXt's GMACs and the named `conv_dw` depthwise 7×7 contributes only 1–2%. Translated back to the convolutional view, the inverted bottleneck (which is just the 1×1 projection up to $4\times$ width and back) accounts for almost the entire GMACs budget and the 7×7 depthwise is a near-free spatial mixer. The 96×96 GMACs cost is modest (0.82 GMACs for ConvNeXt-T), and after the per-family graph rewrite documented in Section 5.7 the resulting native TRT engine runs at 1.95 ms FP16 on the Jetson Orin Nano—in the same band as DeiT3-M (1.92 ms) and only 17% behind the comparably sized Swin-T (1.67 ms). The earlier wall-time penalty often attributed to the un-fused `Transpose`→`LayerNorm`→`Transpose` pattern was a property of the unrewritten ONNX graph (and of ONNX Runtime TRT EP, which falls back to the CUDA EP for that subgraph), not an inherent architectural cost of ConvNeXt itself.

PVT v2 and MobileViT v2 are hybrid. PVT v2's spatial-reduction attention partly suppresses the QKV cost at deep stages, so the family still spends 68–72% of GMACs on the FFN but only ∼10–13% on QKV (vs. ∼25% for DeiT-3 and Swin); the spatial-reduction 1×1/strided convs end up in the *Attn other* category and account for a non-trivial ∼5%. MobileViT v2 is the most distinctive of the 16: convolutional layers (the MobileNet-v2-like stem and the depthwise/pointwise mobile blocks) account for 60% of total GMACs, with separable transformer blocks splitting the remainder between FFN (20%) and the linear projections inside the separable attention (15%). This dual-budget profile is the architectural reason MobileViT v2 places so differently on the Pareto curve from both pure CNNs and pure transformers.

ResNet is conv-bound, as expected. ResNet-D variants spend 91–97% of GMACs in the bottleneck convolutions, with the strided 1×1 downsample shortcuts contributing the small remainder. The conv share *rises* with depth (50d → 200d) because the additional bottleneck stages add convolutions without adding downsample shortcuts. This matches the wall-time profile (98–99% in conv kernels) almost exactly; ResNet-D is the family for which fvcore-measured GMACs are the cleanest predictor of measured Jetson latency.

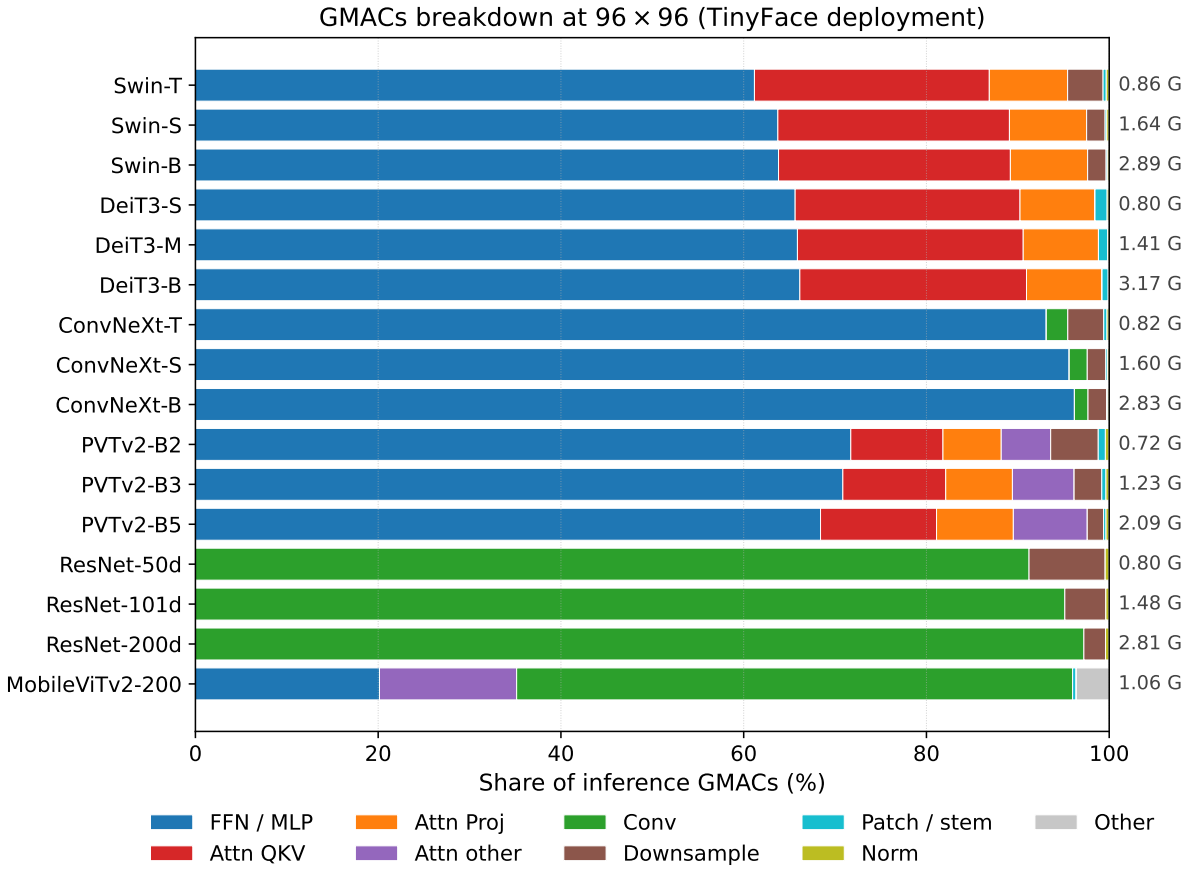


Fig. 15. Per-module GMACs breakdown for the 16 chapter architectures at the 96 × 96 TinyFace deployment resolution. Bars are normalised so that segments express the share of inference GMACs; absolute totals are annotated at the right edge. ConvNeXt's "FFN/MLP" segment is the inverted-bottleneck 1 × 1 pointwise stack, which the timm implementation registers as `nn.Linear`.

Table 7. Top-3 per-module GMACs contributors at the 96×96 TinyFace deployment resolution. "GMACs" is the total backbone cost at batch=1; percentages give each category's share of that total. "FFN / MLP" includes the inverted-bottleneck 1 × 1 projection stack for ConvNeXt; "Attn other" for PVT v2 captures the spatial-reduction projections inside the attention block.

Model	GMACs	1st contributor	2nd contributor	3rd contributor
Swin-T	0.86	FFN / MLP (59%)	Attn QKV (25%)	Attn Proj (8%)
Swin-S	1.64	FFN / MLP (62%)	Attn QKV (25%)	Attn Proj (8%)
Swin-B	2.89	FFN / MLP (63%)	Attn QKV (25%)	Attn Proj (8%)
DeiT3-S	0.80	FFN / MLP (66%)	Attn QKV (25%)	Attn Proj (8%)
DeiT3-M	1.41	FFN / MLP (66%)	Attn QKV (25%)	Attn Proj (8%)
DeiT3-B	3.17	FFN / MLP (66%)	Attn QKV (25%)	Attn Proj (8%)
ConvNeXt-T	0.82	FFN / MLP (93%)	Downsample (4%)	Conv (2%)
ConvNeXt-S	1.60	FFN / MLP (96%)	Downsample (2%)	Conv (2%)
ConvNeXt-B	2.83	FFN / MLP (96%)	Downsample (2%)	Conv (1%)
PVTv2-B2	0.72	FFN / MLP (72%)	Attn QKV (10%)	Attn Proj (6%)
PVTv2-B3	1.23	FFN / MLP (71%)	Attn QKV (11%)	Attn Proj (7%)
PVTv2-B5	2.09	FFN / MLP (68%)	Attn QKV (13%)	Attn Proj (8%)
ResNet-50d	0.80	Conv (91%)	Downsample (8%)	Norm (0%)
ResNet-101d	1.48	Conv (95%)	Downsample (4%)	Norm (0%)
ResNet-200d	2.81	Conv (97%)	Downsample (2%)	Norm (0%)
MobileViTv2-200	1.06	Conv (60%)	FFN / MLP (20%)	Attn other (15%)

Table 7 consolidates the top-3 contributing categories per model at 96 × 96. Because every family in this regime scales linearly with pixels, the percentage shares per category at the canonical 224 × 224 pretraining resolution differ by less than ∼5 pp from the 96 × 96 values; the absolute GMACs simply scale by approximately the pixel ratio $(224/96)^2 \approx 5.4\times$.

The cross-reference between this GMACs view and the wall-time view of Section 5.7 is the real takeaway. Where the two agree (DeiT-3, ResNet-D, ConvNeXt under the rewritten native-TRT engines),

the architecture's compute is well-fused and GMACs are a usable latency proxy. Where they disagree (Swin spends more wall-time fraction in attention than its GMACs fraction would suggest because of the window-partition shuffles), the discrepancy points directly at a deployment-time inefficiency that is not visible in any pure architectural metric. For embedded deployment the practical implication is that *model selection from a published GMACs table is unsafe whenever the target device or runtime cannot fuse the architecture's idiosyncratic non-GMACs operators*—a category that includes essentially every modern ConvNet–Transformer hybrid on TensorRT 10.x for SM 8.7.

5.10 Summary and Implications for Embedded Deployment

Table 8 consolidates the best results across all experiments.

Table 8. Overall best results across all experiments. For each architecture, the best pruning+quantization configuration is shown alongside the unpruned baseline. The highest absolute student accuracy is Swin-T + KD (α=0.0) at 0.616 rank-1; the smallest competitive model is DeiT3-S at 10% heads, INT8 (21.7 MB).

Model	Method	Params	Size	rank-1	mAP
CVLFace teacher (frozen)	Reference	114.9 M	—	0.716	0.664
CVLFace teacher (FT)	Reference	114.9 M	—	0.781	0.744
Training regime optimization					
Swin-T	KD (α=0.0, CVLFace frozen)	27.5 M	106.9 MB	**0.616**	0.664
DeiT3-B	KD (α=0.1, CVLFace FT)	85.8 M	327.7 MB	0.593	0.646
DeiT3-B	Baseline	85.8 M	327.7 MB	0.583	0.639
DeiT3-S	Baseline	21.7 M	83.3 MB	0.552	0.609
Swin-T	Baseline	27.5 M	106.9 MB	0.534	0.594
Swin-B	Baseline	86.7 M	334.9 MB	0.526	0.587
Best pruned + INT8 per architecture					
Swin-T KD (α=0.0)	2 blocks, INT8	24.0 M	24.6 MB	0.583	0.634
DeiT3-S	10% heads, INT8	20.9 M	21.7 MB	**0.565**	0.619
Swin-B	5 blocks, INT8	71.0 M	71.2 MB	0.554	0.612
DeiT3-B KD (α=0.1)	10% heads, INT8	82.9 M	82.0 MB	0.591	0.645
DeiT3-B LoRA	10% heads, INT8	84.7 M	82.0 MB	0.586	0.640
ResNet-50	Baseline	23.5 M	—	0.476	0.542

The experimental findings carry several implications for embedded vision transformer deployment:

1. **Compact transformers outperform large CNNs on low-resolution inputs.** Swin-Tiny (27.5M) outperforms ResNet-200 (62.6M) by +0.066 rank-1 in the baseline setting, and by +0.148 with optimized KD. On TinyFace, where input resolution is severely limited, transformers offer a favorable accuracy–efficiency trade-off. Self-attention's global receptive field is a likely source of this advantage, since it lets the model exploit limited spatial information more effectively than local convolutions; the relative benefit may differ on higher-resolution or non-biometric tasks.
2. **LoRA effectiveness is architecture-dependent.** On TinyFace, LoRA improved all Swin (hierarchical, local-window attention) variants and degraded all DeiT-3 (columnar, global attention) variants. Swin's hierarchical structure may benefit from the regularizing effect of LoRA's low-rank constraint, whereas DeiT-3's global attention layers seem to require higher-rank updates to adapt to this domain. The implication is that PET methods should be evaluated per architecture rather than applied uniformly, and that the LoRA–architecture interaction warrants further study across other tasks and datasets.
3. **Knowledge distillation requires careful tuning of the loss mixing coefficient.** The commonly used $\alpha = 0.7$ leaves significant accuracy on the table. With proper tuning, KD boosts Swin-Tiny from 0.534 (baseline) to 0.616 (a 15.4% relative improvement), making it competitive with models three times its size.
4. **Teacher accuracy on the target task interacts non-trivially with the mixing coefficient.** The higher-accuracy CVLFace teacher enabled pure feature distillation ($\alpha = 0.0$), while the lower-accuracy frozen PETALface teacher required hard-loss anchoring ($\alpha \geq 0.1$) to prevent training collapse. KD configurations should be validated end-to-end rather than relying on teacher accuracy as

a proxy for student improvement, and the optimal α should be re-tuned when changing teachers or target datasets.

5. **Post-training quantization is essentially free.** Across all models tested, FP16 conversion was lossless and INT8 dynamic quantization introduced at most 0.002 absolute rank-1 loss while reducing model size by 3.8–3.9×. For deployment on integer-only accelerators (NPUs, DSPs), INT8 quantization should be a default step rather than a trade-off.
6. **Structured pruning is architecture- and strategy-dependent.** Block dropping improved both Swin variants (Swin-T +0.010, Swin-B +0.028 rank-1 from baseline) and was the only safe pruning method for KD-distilled Swin-T, while head pruning degraded Swin at every ratio tested. For DeiT3 models, conservative head pruning (10%) improved DeiT3-S (+0.013 rank-1) and preserved DeiT3-B KD accuracy (0.591 ≈ 0.593). The pruning axis (depth vs. width) needs to be matched to both the architecture and the training regime, and automated pruning search may unlock further gains.
7. **KD and pruning interact in non-obvious ways.** The strongest KD checkpoint (Swin-T, α=0.0, rank-1 = 0.616) suffers catastrophic degradation under head pruning (−0.213 at 10%) even though the same pruning is benign on the weaker baseline (0.534 → 0.500). Distillation with α=0.0 (pure feature matching) appears to concentrate knowledge into fewer heads, which makes the model fragile under head removal. Practitioners planning to combine KD with pruning should either use moderate α values or prefer block dropping to preserve distilled representations.
8. **Pruning and quantization compose without compounding losses.** INT8 quantization of pruned models induced the same ∼0.000–0.003 rank-1 change as on unpruned models, so both compression techniques can be applied sequentially. The combination of pruning+INT8 achieves 3.8–5.1× compression while maintaining or improving accuracy in several configurations.

5.11 Power-Mode Trade-offs

All latency numbers reported above were measured under the Jetson Orin Nano's `MAXN_SUPER` power mode, which removes the on-board wattage cap. Practical embedded deployments, however, often run under a tighter thermal or battery budget. To quantify how the previously reported throughput scales when this budget is reduced, we re-benchmarked a representative subset of fourteen (model, precision) configurations across the three power modes exposed by `nvpmodel` on the Orin Nano "Super" firmware (R36.4.3): **15W** (ID 0), **25W** (ID 1) and **MAXN_SUPER** (ID 2). Between modes we issued `nvpmodel -m <id>` followed by `jetson_clocks` and a five-second settling sleep so that DVFS, fan and voltage rails reached steady state before the timing window opened; all other `trtexec` arguments (workspace, warmup, iterations, CUDA-graph capture, batch size) were held identical so that the only varying factor is the runtime power cap. All fourteen configurations—including ConvNeXt-T and PVTv2-B2, which were re-exported using the per-family rewrites of Section 5.7—were benchmarked through the same unified native-TRT harness.

Sustained wall power was captured per mode with `tegrastats --interval 500` and parsed for the `VDD_IN` rail. The measured averages diverge meaningfully from the configured budgets: the 15W mode draws 7.6 W mean / 13.0 W p95, the 25W mode draws 12.4 W mean / 15.0 W p95, and `MAXN_SUPER` draws 13.5 W mean / 16.0 W p95. Notably, `MAXN_SUPER` only adds about 1 W of sustained draw over the 25W cap while still producing the headline throughput. Under a single-stream batch=1 workload at 96 × 96 the Orin Nano never approaches the theoretical 25 W envelope, so removing the cap is essentially free in this regime. Table 9 reports mean latency, throughput and the relative 15W slowdown for each model; Figure 16 visualises the latency bars and Figure 17 normalises throughput by the *configured* budget so that the 15W mode is given credit for its lower power envelope.

Discussion. The sweep results break down into three patterns:

- **Graceful degradation under 15W.** The throughput coefficient of variation across the three modes (*FPS CV* column in Table 9) stays in a narrow 0.10–0.15 band: no model "collapses" under the 15W cap. The smallest variance (∼0.10) is observed for the DeiT3-B family (DeiT3-B baseline, KD α=0.1 and LoRA-pruned), making them the most "power-mode-stable" configurations in the suite. This is consistent with the per-layer profiling of Section 5.4: DeiT3-B's runtime is dominated by FFN GEMMs that already saturate the GPU SMs at the 15W clock cap, leaving little additional throughput to extract at higher clocks.
- **Lightweight workloads scale most aggressively with the budget.** The largest 15W slowdowns are seen on PVTv2-B2 (1.40×), ConvNeXt-T (1.33×), DeiT3-S (1.32×), Swin-T-KD-blocks 2 (1.30×) and Swin-T-KD α=0 (1.30×): exactly the models whose per-iteration kernels are short enough to be

Table 9. Latency, throughput and 15W slowdown across the three Jetson Orin Nano power modes for the 14-model representative subset (batch=1, 96 × 96). All engines benchmarked under the unified native TensorRT harness (`trtexec --useCudaGraph`); ConvNeXt-T and PVTv2-B2 use the rewritten engines described in Section 5.7. *Slowdown* is the 15W mean latency divided by the `MAXN_SUPER` mean latency; *FPS CV* is the coefficient of variation of throughput across the three modes (lower = more power-mode-stable).

		Mean ms			FPS			Summary	
Model	Prec	15W	25W	MAXN	15W	25W	MAXN	15W slow.	FPS CV
deit3_base	fp16	4.16	3.53	3.46	240	283	288	1.20×	0.098
deit3_base	int8	4.16	3.52	3.46	240	284	289	1.20×	0.099
deit3_small	fp16	1.68	1.32	1.27	594	754	786	1.32×	0.145
resnet200d	fp16	4.51	3.57	3.42	222	280	293	1.32×	0.143
resnet50d	fp16	1.53	1.23	1.19	653	808	837	1.28×	0.129
swin_base	fp16	5.00	4.17	4.06	200	240	246	1.23×	0.109
swin_tiny	fp16	2.13	1.74	1.66	468	575	600	1.28×	0.127
swin_tiny	int8	2.13	1.72	1.66	469	579	602	1.28×	0.129
swin_tiny_KD_α=0	fp16	2.15	1.72	1.66	464	581	602	1.30×	0.135
deit3_base_KD_α=0.1	fp16	4.16	3.52	3.47	240	284	288	1.20×	0.097
deit3_base_LoRA-heads-10%	fp16	4.06	3.45	3.39	246	289	295	1.20×	0.096
swin_tiny_KD-blocks-2	fp16	1.87	1.50	1.44	534	666	693	1.30×	0.135
Native TRT engines for the previously TRT-incompatible architectures (re-exported via the per-family rewrites of Section 5.7):									
convnext_tiny	fp16	2.60	2.04	1.95	385	490	512	1.33×	0.147
pvt_v2_b2	fp16	3.84	2.91	2.74	260	343	365	1.40×	0.171
Sustained `VDD_IN` wall power per mode (`tegrastats`, mean / p95):									
15W: 7.61 W / 13.03 W 25W: 12.36 W / 15.04 W MAXN_SUPER: 13.46 W / 15.97 W									

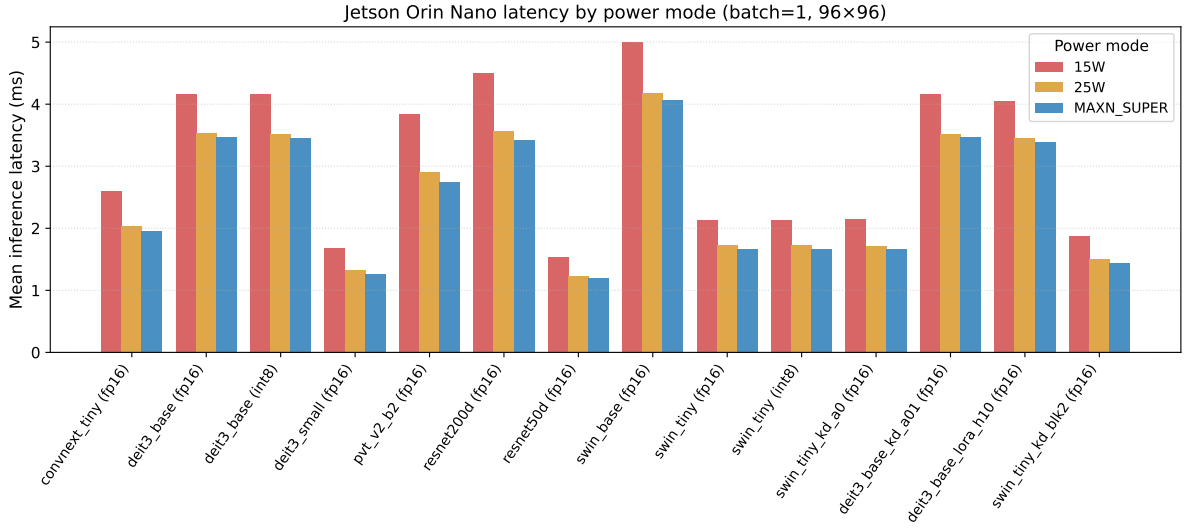


Fig. 16. Mean inference latency (ms, batch=1, 96 × 96) for the 14-model representative subset across the three Jetson Orin Nano power modes, all under the unified native TRT harness. The 15W mode adds a uniform 20–40% latency penalty over `MAXN_SUPER`; 25W is within 2–6% of `MAXN_SUPER` for every model, mirroring the modest ∼1 W difference observed via `tegrastats`.

limited by GPU clock rather than by SM occupancy. PVTv2-B2's especially clock-sensitive behaviour is consistent with its layer-norm-heavy graph, even after the rewrite to native TRT.

– **Configured budget vs. measured wall power.** The configured 15W cap delivers a measured 7.6 W mean / 13.0 W p95, i.e. roughly half the nameplate value, because the workload never simultaneously stresses all six A78 cores and the GPU. Conversely, `MAXN_SUPER` only draws ∼1 W more than the 25W mode but yields a 2–5% throughput uplift; outside of strictly thermally constrained enclosures, leaving the device in `MAXN_SUPER` is the correct default. The discrepancy between the `nvpmodel` label and the measured `VDD_IN` draw is a reminder that the configured budget is an upper bound, not an estimate of actual consumption.

Deployment recommendation per power budget. For a sustained ≤10 W battery-class envelope, the 15W `nvpmodel` setting paired with Swin-T-KD-blocks 2 (FP16) delivers 533 FPS at 1.87 ms while staying inside a 7.6 W mean draw—the best accuracy/latency/power combination in the table. For a 15–20 W passively-

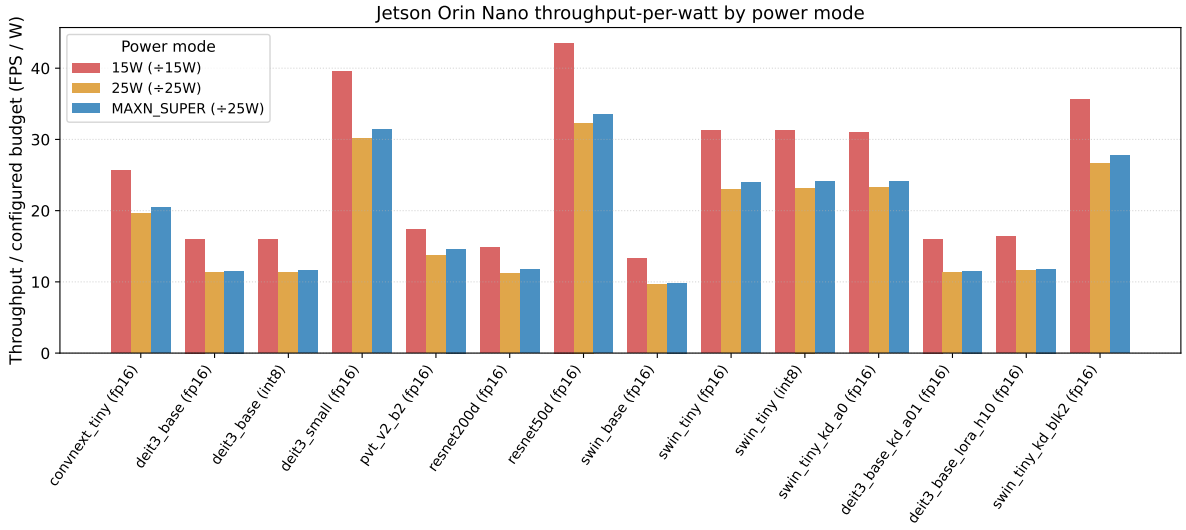


Fig. 17. Throughput normalised by the *configured* `nvpmodel` budget (15, 25 or 25 W). On a per-watt basis, the 15W mode is competitive with or wins against `MAXN_SUPER` for every workload, despite delivering lower absolute FPS.

Table 10. Recommended model configurations for common embedded deployment scenarios, based on the experimental results from this study. All latency figures are FP16 on the Jetson Orin Nano at batch size 1. INT8 sizes assume ONNX dynamic quantization.

Scenario	Recommended Model	rank-1	Latency	Size
Maximum accuracy	DeiT3-B KD (α=0.1), 10% heads, FP16	0.591	2.6 ms	164 MB
Best accuracy–speed	Swin-T KD (α=0.0), 2 blk drop, FP16	0.583	∼1.3 ms	49 MB
Minimum latency	ResNet-50, INT8 (TRT native)	0.476	0.6 ms	24 MB
Minimum memory	DeiT3-S, 10% heads, INT8	0.565	1.0 ms	21.7 MB

cooled envelope, the 25W mode captures 96–98% of `MAXN_SUPER`'s throughput at ∼1 W lower sustained draw and is the recommended default. For actively-cooled or wall-powered deployments, `MAXN_SUPER` should be used because its 1 W premium over 25W mode is well below the noise floor of the platform's power supply.

Deployment Decision Guide. Table 10 summarises the experimental results as a set of concrete recommendations organised by deployment priority. The recommendations are specific to the TinyFace task (96×96 input, face identification) on the Jetson Orin Nano; deployments on other tasks, input resolutions, or hardware should be validated empirically.

The first scenario—maximum accuracy with moderate size—is best served by DeiT3-B with optimized KD and conservative head pruning, delivering 0.591 rank-1 at 164 MB (FP16) or 82 MB (INT8). The second scenario—best accuracy–speed trade-off—favors Swin-T with KD and block dropping, which achieves 0.583 rank-1 while fitting in 49 MB at FP16. For strict latency-critical applications (e.g., real-time video at 30+ FPS), ResNet-50 with TRT-native INT8 achieves 0.63 ms (∼1,590 FPS) at the cost of lower accuracy (0.476 rank-1). Finally, for the most memory-constrained deployments (e.g., microcontrollers or shared-memory multi-model systems), DeiT3-S with 10% head pruning at INT8 achieves the smallest footprint (22 MB) while still exceeding its unpruned baseline accuracy (0.565 vs. 0.552).

Two limitations of this analysis should be noted. First, all experiments were conducted on a single dataset (TinyFace) with a specific input resolution (96×96); the relative performance of architectures may differ on higher-resolution inputs or non-biometric tasks. Second, latency measurements were obtained on a single hardware platform (Jetson Orin Nano with TensorRT 10.3); different accelerators (e.g., Qualcomm Hexagon DSP, Apple Neural Engine) may produce substantially different rankings due to architecture-specific kernel optimizations [30].

6 Conclusion

This chapter investigated the full design space for deploying vision transformers on resource-constrained embedded systems, using face identification on the TinyFace benchmark [4] as a case study. Starting from 16 model variants across six architecture families, the study systematically evaluated parameter-efficient fine-tuning via LoRA [12], feature-level knowledge distillation with two teacher configurations, structured pruning (head removal and block dropping), and post-training quantization (FP16, INT8)—producing over 100 model configurations that were benchmarked for both biometric accuracy and on-device inference latency on an NVIDIA Jetson Orin Nano. The resulting analysis connects architectural design choices to measurable deployment outcomes and identifies several findings with practical implications for embedded vision systems.

The clearest finding is that on this low-resolution task, architecture choice dominates the accuracy–efficiency trade-off. Compact vision transformers consistently outperform larger CNNs: Swin-Tiny (27.5M parameters) exceeds ResNet-200 (62.6M) by +0.066 rank-1 in the baseline setting, and with optimally tuned knowledge distillation ($\alpha = 0.0$, frozen CVLFace teacher), Swin-Tiny reaches 0.616 rank-1—an absolute gain of +0.082 over its baseline (a 15.4% relative improvement) that makes a 27.5M-parameter model competitive with unpruned 86M-parameter alternatives (Table 8). Self-attention's global receptive field appears to be a key factor when discriminative features must be extracted from severely resolution-limited inputs (96×96 from $\sim 30 \times 30$ native). Architecture choice also determines which optimization techniques are effective: LoRA improved all Swin variants (+0.010 to +0.028 absolute rank-1) while degrading all DeiT-3 variants (−0.025 to −0.037), and head pruning was well-tolerated by DeiT-3 but catastrophic for KD-distilled Swin models. Optimization techniques are not interchangeable black-box tools, and these architecture-dependent interactions need to be tested per family.

Model compression proved effective when matched to the architecture. Post-training INT8 quantization is essentially cost-free, introducing at most 0.002 absolute rank-1 loss while delivering 3.8–3.9× size reduction across all 24 pruned models tested. Structured pruning, when the pruning axis is aligned with the architecture, can maintain or even *improve* accuracy: block dropping improved Swin-T (+0.010) and Swin-B (+0.028) over their baselines, confirming systematic depth over-parameterization on this task, while 10% head pruning improved DeiT3-S (+0.013 to 0.565). Pruning and quantization compose without compounding losses, enabling combined compression ratios of 3.8–5.1× (Table 5). Knowledge distillation interacts with pruning in less obvious ways: the strongest KD checkpoint (Swin-T, α=0.0, rank-1 = 0.616) suffered catastrophic degradation under head pruning (−0.213) while block dropping remained safe (0.583). The implication is that pure feature distillation concentrates learned representations into fewer heads and creates a fragility that should be anticipated when designing a compression pipeline.

Hardware-in-the-loop evaluation on the Jetson Orin Nano showed that native TensorRT compatibility is architecture-dependent but, for every architecture in this study, recoverable through a principled module-level graph rewrite. Models that apply layer normalization to NCHW spatial feature maps—ConvNeXt, PVT-v2, and MobileViT-v2—initially failed `trtexec` conversion entirely, due to missing SM 8.7 kernels for 4-D normalization patterns. Several intuitive workarounds (opset decomposition, a custom NCHW normalization module, post-hoc ONNX graph surgery) all fail because TensorRT's Myelin compiler re-fuses the rewritten arithmetic back into the same unsupported `ForeignNode`. The fix that does work is per-family and module-level: ConvNeXt's `LayerNorm2d` is replaced with a wrapper that reshapes to (B, HW, C) around a standard `nn.LayerNorm`, combined with timm's `conv_mlp=True` backbone; PVT-v2's fused-attention path is disabled before export to expose a clean opset-17 graph; and MobileViT-v2's single-group `GroupNorm` is replaced by a manual elementwise decomposition. All three rewrites reproduce the original PyTorch outputs to cosine similarity ≥ 0.999 and produce `trtexec`-compilable engines, allowing all sixteen architectures to be benchmarked under a single uniform harness. Under that harness the previously reported 4.8× Swin-T vs. ConvNeXt-T gap (1.72 ms vs. 4.53 ms FP16) shrinks to 1.17× (1.67 ms vs. 1.95 ms), and the ONNX Runtime TensorRT EP fallback—a viable workaround when a per-family rewrite is not feasible—incurs a measurable 1.5–2.5× latency penalty relative to the rewritten native engines (e.g. ConvNeXt-T 4.53 ms vs. 1.95 ms FP16, MobileViT-v2-200 5.05 ms vs. 1.90 ms). Per-layer profiling (Figure 13) further showed that DeiT-3 models are FFN-dominated (68–83% of compute), Swin models distribute compute more evenly between attention and FFN (27–31% attention), and ResNet is almost entirely convolution-bound (98–99%). These profiles explain why block dropping is effective for Swin (it removes both attention and FFN proportionally) while head pruning has limited latency impact (attention accounts for less than a third of total time).

Several practical challenges shaped the experimental design. Training with the AdaFace margin-based loss [28] required careful hyperparameter tuning per architecture family to avoid numerical instability, particularly for larger models. The extreme input resolution mismatch between TinyFace's native $\sim 30 \times 30$ imagery and the models' expected input size meant that all models operated on heavily interpolated inputs, placing a ceiling on achievable accuracy regardless of model capacity. The Jetson Orin Nano's 8 GB unified memory constrained batch sizes during evaluation and required exclusive use of FP16 or INT8 precisions for larger models, reinforcing the practical importance of the quantization and pruning pipeline developed in this work.

For practitioners deploying vision transformers on embedded platforms, the experimental results support several concrete recommendations. When accuracy is the primary objective, DeiT3-B with optimized KD (α=0.1) and 10% head pruning at FP16 delivers 0.591 rank-1 at 2.6 ms inference latency. When balancing accuracy against size and speed, Swin-T with KD (α=0.0) and block dropping at INT8 achieves 0.583 rank-1 at 24.6 MB. For the most memory-constrained deployments, DeiT3-S with 10% head pruning at INT8 fits in 21.7 MB while exceeding its unpruned baseline (0.565 vs. 0.552). Table 10 provides a consolidated decision matrix organized by deployment priority. In all cases, practitioners should validate native TRT compatibility early in the development cycle; for architectures that fail `trtexec` out of the box, the per-family module-level rewrites described in Section 5.7 restore a uniform native-TRT deployment path with cosine similarity ≥ 0.999 to the original PyTorch model.

Several directions remain for future work. Joint knowledge distillation and pruning—optimizing both simultaneously rather than sequentially—may yield further improvements and potentially avoid the fragility observed when pruning KD-distilled models. Automated pruning search across both depth and width axes simultaneously could identify configurations that manual grid search misses [17]. Extending this analysis to additional edge platforms (Qualcomm Hexagon DSP, Apple Neural Engine, MediaTek APU) would reveal how architecture rankings change with hardware-specific kernel support [30]. Newer efficient transformer architectures such as EfficientViT [23] and FastViT [40], which incorporate hardware-aware design principles from their inception, may shift the Pareto front established here. Finally, evaluating these findings on higher-resolution biometric tasks and non-biometric vision domains would help establish the generality of the architecture-dependent optimization patterns observed on TinyFace.

The broader picture is that vision transformers are now a practical option for embedded deployment, not just a research subject. The relevant question is no longer whether transformers can run on edge devices, but how to navigate the design space of architectures, training regimes, and compression techniques to find configurations that meet specific deployment constraints. The architecture-aware methodology presented in this chapter is intended as a starting template for that process, and we expect it to remain useful as both transformer architectures and embedded hardware continue to evolve.

References

1. Ambient Scientific: Training vs inference: Training teaches, inference delivers. Ambient Scientific Blog (2025), `https://www.ambientscientific.ai/blogs/training-vs-inference-training-teaches-inference-delivers`
2. Ba, J.L., Kiros, J., Hinton, G.: Layer normalization. arXiv:1607.06450 (2016)
3. Bolya, D., Ryali, C., Hoffman, J., Feichtenhofer, C.: Window attention is bugged: How not to interpolate position embeddings. In: International Conference on Learning Representations (ICLR) (2024)
4. Cheng, Z., Zhu, X., Gong, S.: Low-resolution face recognition. In: Computer Vision – ACCV 2018 (2018), `https://arxiv.org/abs/1811.08965`
5. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT). pp. 4171–4186 (2019), `https://aclanthology.org/N19-1423/`
6. Dorszewski, T., Tětková, L., Jenssen, R., Hansen, L.K., Wickstrøm, K.K.: From colors to classes: Emergence of concepts in vision transformers. In: Proceedings of the 3rd World Conference on Explainable Artificial Intelligence (xAI) (2025), arXiv:2503.24071
7. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An image is worth 16x16 words: Transformers for image recognition at scale. In: International Conference on Learning Representations (ICLR) (2021), `https://openreview.net/forum?id=YicbFdNTTy`
8. Gherasim, I., García Sánchez, C.: Hardware optimization on android for inference of ai models. arXiv:2511.13453 (2025)

9. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 770–778 (2016). https://doi.org/10.1109/CVPR.2016.90, https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html
10. Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network. arXiv:1503.02531 (2015)
11. Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., Gelly, S.: Parameter-efficient transfer learning for NLP. In: Proceedings of the 36th International Conference on Machine Learning (ICML) (2019)
12. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: Lora: Low-rank adaptation of large language models. In: International Conference on Learning Representations (ICLR) (2022), https://arxiv.org/abs/2106.09685
13. IBM Cloud Education: What is quantization aware training? IBM Developer Blog (2023), https://www.ibm.com/think/topics/quantization-aware-training
14. Ignatov, A., Timofte, R., Chou, W., Wang, K., Wu, M., Hartley, T., Van Gool, L.: Ai benchmark: Running deep neural networks on android smartphones. In: European Conference on Computer Vision (ECCV) Workshops (2019), arXiv:1810.01109
15. Jiang, B., Chen, J., Liu, Y.: Single-shot pruning and quantization for hardware-friendly neural network acceleration. Engineering Applications of Artificial Intelligence **126**, 106816 (2023)
16. Kopetz, T.: Transformer models and NPU IP co-optimized for the edge. CEVA Blog (2023)
17. Kwon, W., Kim, S., Mahoney, M.W., Hassoun, J., Keutzer, K., Gholami, A.: A fast post-training pruning framework for transformers. In: Advances in Neural Information Processing Systems (NeurIPS) (2022), arXiv:2204.09656
18. Li, Y., Yuan, G., Wen, Y., Hu, J., Evangelidis, G., Tulyakov, S., Wang, Y., Ren, J.: Efficientformer: Vision transformers at mobilenet speed. In: Advances in Neural Information Processing Systems (NeurIPS) (2022), https://arxiv.org/abs/2206.01191
19. Li, Z., Gu, Q.: I-ViT: Integer-only quantization for efficient vision transformer inference. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). pp. 17019–17029 (2023)
20. Li, Z., Paolieri, M., Golubchik, L.: A study on inference latency for vision transformers on mobile devices. In: Proceedings of VALUETOOLS 2024. Springer (2025). https://doi.org/10.48550/arXiv.2510.25166, arXiv:2510.25166
21. Lin, Y., Zhang, T., Sun, P., Li, Z., Zhou, S.: FQ-ViT: Post-training quantization for fully quantized vision transformer. In: Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI). pp. 1172–1178 (2022)
22. Liu, D., Zhu, Y., Liu, Z., Liu, Y., Han, C., Tian, J., Li, R., Yi, W.: A survey of model compression techniques: past, present, and future. Frontiers in Robotics and AI **12** (2025). https://doi.org/10.3389/frobt.2025.1518965, https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2025.1518965/full
23. Liu, X., Peng, H., Zheng, N., Yang, Y., Hu, H., Yuan, Y.: EfficientViT: Memory efficient vision transformer with cascaded group attention. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2023), https://arxiv.org/abs/2305.07027
24. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B.: Swin transformer: Hierarchical vision transformer using shifted windows. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) (2021), https://arxiv.org/abs/2103.14030
25. Liu, Z., Mao, H., Wu, C.Y., Feichtenhofer, C., Darrell, T., Xie, S.: A convnet for the 2020s. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2022), https://arxiv.org/abs/2201.03545
26. Mehta, S., Rastegari, M.: Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer. In: International Conference on Learning Representations (ICLR) (2022), https://arxiv.org/abs/2110.02178
27. Nadalini, D., Rusci, M., Palossi, D.: Multi-modal on-device learning for monocular depth estimation on ultra-low-power MCUs. arXiv:2512.00086 (2025)
28. Narayan, K., Nair, N.G., Xu, J., Chellappa, R., Patel, V.M.: Petalface: Parameter efficient transfer learning for low-resolution face recognition. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) (2025), https://openaccess.thecvf.com/content/WACV2025/html/Narayan_PETALface_Parameter_Efficient_Transfer_Learning_for_Low-Resolution_Face_Recognition_WACV_2025_paper.html
29. Narayan, K., Patel, V.M.: Facemoe: Mixture of experts for low-resolution face recognition (2025), https://kartik-3004.github.io/portfolio/papers/FaceMOE.pdf, preprint, under review
30. Ngo, D., Park, H.C., Kang, B.: Edge intelligence: A review of deep neural network inference in resource-limited environments. Electronics **14**(12), 2495 (2025)
31. Quadric: Vision transformers (vit) on npus: New requirements for on-device ai chips. https://quadric.ai/blog/vision-transformers-vit-npu-on-device-ai (2023)
32. Ranftl, R., Bochkovskiy, A., Koltun, V.: Vision transformers for dense prediction. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) (2021)

33. Ranjan, N., Savakis, A.: Mix-QViT: Mixed-precision vision transformer quantization driven by layer importance and quantization sensitivity. arXiv:2501.06357 (2025)
34. Rodrigo, M., Cuevas, C., García, N.: Comprehensive comparison between vision transformers and convolutional neural networks for face recognition tasks. Scientific Reports **14**, 21392 (2024). https://doi.org/10.1038/s41598-024-72254-w, https://www.nature.com/articles/s41598-024-72254-w
35. Roth, W., Zapp, L.C., Störf, M., Uhl, A.: Resource-efficient neural networks for embedded systems. Journal of Machine Learning Research **25**(188), 1–51 (2024)
36. Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A.C., Fei-Fei, L.: ImageNet large scale visual recognition challenge. International Journal of Computer Vision (IJCV) **115**(3), 211–252 (2015). https://doi.org/10.1007/s11263-015-0816-y
37. Saha, S., Xu, L.: Vision transformers on the edge: A comprehensive survey of model compression and acceleration strategies. Neurocomputing (2025). https://doi.org/10.1016/j.neucom.2025.130417
38. Shang, Y., Liu, G., Kompella, R.R., Yan, Y.: Enhancing post-training quantization calibration through contrastive learning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops (2024)
39. Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., Jégou, H.: Training data-efficient image transformers & distillation through attention. In: Proceedings of the 38th International Conference on Machine Learning (ICML). vol. 139, pp. 10347–10357 (2021), https://proceedings.mlr.press/v139/touvron21a.html
40. Vasu, P.K.A., Gabriel, J., Zhu, J., Tuzel, O., Ranjan, A.: FastViT: A fast hybrid vision transformer using structural reparameterization. In: IEEE/CVF International Conference on Computer Vision (ICCV) (2023)
41. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS) (2017), https://arxiv.org/abs/1706.03762
42. Wang, W., Xie, E., Li, X., Fan, D.P., Song, K., Liang, D., Lu, T., Luo, P., Shao, L.: Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) (2021), https://openaccess.thecvf.com/content/ICCV2021/html/Wang_Pyramid_Vision_Transformer_A_Versatile_Backbone_for_Dense_Prediction_Without_ICCV_2021_paper.html
43. Wang, Y., Dong, S., Huang, Z., You, Y., He, L., Yang, H., Liu, Y., Jia, H.: HyC-LoRA: Memory-efficient lora fine-tuning with hybrid activation compression. In: Proceedings of Machine Learning and Systems (MLSys) (2025)
44. Wightman, R.: PyTorch Image Models. https://github.com/huggingface/pytorch-image-models (2019). https://doi.org/10.5281/zenodo.4414861
45. Wu, K., Zhang, J., Peng, H., Liu, M., Xiao, B., Fu, J., Yuan, L.: Tinyvit: Fast pretraining distillation for small vision transformers. In: Computer Vision – ECCV 2022 (2022), https://arxiv.org/abs/2207.10666
46. Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., Liu, T.Y.: On layer normalization in the transformer architecture. In: Proceedings of the 37th International Conference on Machine Learning (ICML) (2020)
47. Zhang, A., Lipton, Z.C., Li, M., Smola, A.J.: Dive into Deep Learning. Cambridge University Press, Cambridge (2023), https://d2l.ai
48. Zhang, J., Peng, H., Wu, K., Liu, M., Xiao, B., Fu, J., Yuan, L.: Minivit: Compressing vision transformers with weight multiplexing. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2022), https://openaccess.thecvf.com/content/CVPR2022/html/Zhang_MiniViT_Compressing_Vision_Transformers_With_Weight_Multiplexing_CVPR_2022_paper.html
49. Zhang, T., et al.: On-device learning. In: Machine Learning Systems Design. Harvard University, Cambridge, MA (2023)
50. Zhao, Y., Chen, J., Zhang, S.Q., Sarwar, S.S., Stangherlin, K.H., Gomez, J.T., Seo, J.s., De Salvo, B., Liu, C., Gibbons, P.B., Li, Z.: H4h: Hybrid convolution-transformer architecture search for npu-cim heterogeneous systems for ar/vr applications. In: Proceedings of the 30th Asia and South Pacific Design Automation Conference (ASPDAC) (2025)
51. Zhu, S., Voigt, T., Ko, J., Rahimian, F.: On-device training: A first overview on existing systems. ACM Transactions on Sensor Networks (2024). https://doi.org/10.1145/3696003